



Appunti di Analisi e Progettazione del Software

Nicholas Scorrano

Anno Accademico 2025/2026

Indice

1	Modello a cascata	5
1.1	Cosa è?	5
1.2	Le fasi del modello	5
1.2.1	Analisi dei Requisiti	5
1.2.2	Progettazione	5
1.2.3	implementazione	5
1.2.4	Collaudo	6
1.2.5	Rilascio e Manutenzione	6
1.3	Caratteristiche chiave	6
1.3.1	Plan-driven	6
1.3.2	Specializzazione	6
1.3.3	Rigidità	6
1.4	Vantaggi	6
1.4.1	Semplicità e chiarezza	6
1.4.2	Ideale per requisiti stabili	6
1.5	Svantaggi	6
1.5.1	Incapacità di gestire i cambiamenti	6
1.5.2	Feedback tardivo	7
1.5.3	Debito di integrazione	7
2	UP - Unified Process	8
2.1	Cosa è?	8
2.2	Le 3 caratteristiche fondamentali dell'UP	8
2.2.1	Pilotato dai casi d'uso	8
2.2.2	Guidato dai fattori di rischio	8
2.2.3	Incentrato sull'architettura	8
2.3	Le 4 fasi del ciclo di vita	8
2.3.1	Ideazione	9
2.3.2	Elaborazione	9
2.3.3	Costruzione	9
2.3.4	Transizione	9
2.4	Dinamica di Sviluppo: Iterazioni, Discipline e incrementi	9
2.4.1	Iterazioni	9
2.4.2	Variazione dell'impegno	9
2.4.3	Release e incremento	9
2.5	Vantaggi	10
2.6	Svantaggi	10
3	Agile	11
3.1	Cosa è?	11
3.2	Valori e Principi	11
3.3	Flusso di valore: Sprint e Artefatti	11
3.3.1	Product Backlog	11
3.3.2	Sprint (iterazione)	11
3.3.3	Daily Scrum	12
3.3.4	Incremento Potenzialmente Rilasciabile	12
3.3.5	Velocity	12
3.4	Vantaggi	12

3.5	Svantaggi	12
4	SCRUM	13
4.1	Cosa è?	13
4.2	Ruoli in Scrum	13
4.2.1	Product Owner	13
4.2.2	Team di Sviluppo	13
4.2.3	ScrumMaster	13
4.3	Elaborati	13
4.3.1	Product Backlog	13
4.3.2	Sprint Backlog	14
4.3.3	Incremento Potenzialmente Rilasciabile	14
4.4	Ciclo di vita	14
4.4.1	Sprint	14
4.4.2	Velocity	14
4.4.3	Eventi chiave	14
4.5	Vantaggi	14
4.5.1	flessibilità e Adattabilità	14
4.5.2	Motivazione e Visibilità	14
4.6	Svantaggi	15
4.6.1	Il mito dell'incremento "Shippable"	15
4.6.2	Problemi di scalabilità	15
5	Architettura Logica	16
5.1	Cosa è l'Architettura Logica?	16
5.2	Architettura a Strati	16
5.2.1	Cosa è uno stato?	16
5.2.2	Architettura a strati stretta	16
5.2.3	Architettura a strati rilassata	16
5.3	Cosa è un architettura software?	17
6	Diagramma dei Package	18
6.1	A cosa serve?	18
6.2	Definizioni: Livelli, Strati e Partizioni	18
6.3	Progettazione degli strati	19
6.3.1	Gerarchia degli stati	19
6.3.2	Scelta comune per gli strati	19
6.3.3	Principio di separazione Modello-Vista	20
7	Diagramma della Macchina a Stati	21
7.1	A cosa servono?	21
7.2	Sintassi base	21
7.3	Stati	21
7.3.1	Definizione	21
7.3.2	Sintassi	21
7.4	Eventi	22
7.4.1	Definizione	22
7.4.2	Tipologie di eventi	22
7.5	Stati Composti	24
7.5.1	Definizione	24
7.5.2	Stati Composti Semplici (Non-Ortogonal)	24
7.5.3	Stati Composti Ortogonali	24
7.6	Sottomacchine	25
7.6.1	Cosa sono?	25

7.6.2	Stati della Sotto Macchina	25
7.6.3	Sintassi degli stati della Sotto-Macchine	25
7.6.4	Comunicazione tra sotto-macchine	26
7.6.5	Comunicazione tramite stato di sync	26
7.6.6	Stati con memoria semplice	27
7.6.7	Sottostato con memoria multilivello	27
7.7	Esempio: Elabora vendita	28
8	Pattern GRASP	29
8.1	Definizione	29
8.1.1	Approccio RDD	29
8.1.2	Principi fondamentali	29
8.1.3	Relazione tra gli elaborati	30
8.2	Rappresentazione di un Pattern GRASP	30
9	Pattern GRASP di base	31
9.1	Creator	31
9.1.1	Definizione	31
9.1.2	Esempio diagramma delle classi	31
9.1.3	Esempio diagramma di sequenza di stato	31
9.2	Information Expert	32
9.2.1	Definizione	32
9.2.2	Esempio diagramma delle classi	32
9.2.3	Esempio diagramma di sequenza di stato	32
9.3	Low Coupling	33
9.3.1	Definizione	33
9.3.2	Effetto di Information Expert	33
9.3.3	Esempio di alto accoppiamento	33
9.4	Controller	34
9.4.1	Definizione	34
9.4.2	Possibili approcci	34
9.4.3	Esempio MonopoluGame	34
9.5	High Cohesion	35
9.5.1	Definizione	35
10	Patter GRASP Avanzati	36
10.1	Pure Fabricator	36
10.1.1	Definizione	36
10.1.2	Esempio POS NextGen	36
10.2	Polymorphism	37
10.2.1	Definizione	37
10.2.2	Esempio POS NextGen	37
10.2.3	Esempio Monopoly	37
10.3	Indirection	38
10.3.1	Definizione	38
10.3.2	Esempio POS NextGen	38
10.4	Protected Variations	39
10.4.1	Definizione	39
11	Design Pattern GoF	40
11.1	Cosa sono?	40
11.2	Tabella	40
11.3	Adapter	41
11.3.1	Definizione	41

11.3.2	Esempio POS NextGen	41
11.4	Factory	42
11.4.1	Definizione	42
11.5	Singleton	43
11.5.1	Definizione	43
11.6	Strategy	44
11.6.1	Definizione	44
11.6.2	Esempio diagramma delle classi	44
11.6.3	Esempio diagramma di sequenza	44
11.7	Composite	45
11.7.1	Definizione	45
11.7.2	Diagramma delle classi	45
11.8	Facade	46
11.8.1	Definizione	46
11.8.2	Diagramma delle classi	46
11.8.3	Diagramma dei Package	46
11.9	Observer	47
11.9.1	Definizione	47
11.9.2	Diagramma delle Classi	47
11.9.3	Diagramma di Sequenza	47
12	TDD - Test Driven Development	48
12.1	Definizione	48
12.2	Cosa è un test unitario?	48
12.2.1	Schema di test	48
12.2.2	Tipi di test	48
12.3	Vantaggi	48
12.4	Svantaggi	49
12.5	Progettazione degli input	49
13	Refactoring	50
13.1	Definizione	50
13.2	Obiettivi e Vantaggi	50
13.3	Quando usarlo?	50
13.4	Procedura	50
13.5	Cosa è un Code Smell?	51
13.6	Tabella Code Smell	51
14	Visibilità	53
14.1	Definizione	53
14.2	Tipi di visibilità	53
14.2.1	Visibilità per attributo	53
14.2.2	Visibilità per parametro	53
14.2.3	Visibilità locale	53
14.2.4	Visibilità globale	53
14.3	Trasformazione della visibilità	54
14.3.1	Da visibilità locale a visibilità per parametro	54
14.3.2	Da visibilità per parametro a visibilità per attributo	54

1. Modello a cascata

1.1 Cosa è?

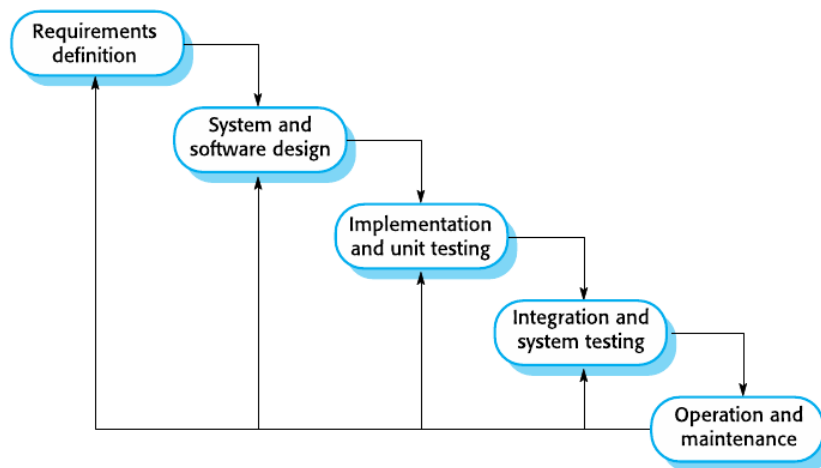
Il modello a cascata rappresenta storicamente il primo approccio sistematico e disciplinato allo sviluppo del software.

È un modello di processo lineare e sequenziale, dove lo sviluppo scorre verso il basso attraverso diverse fasi, proprio come l'acqua di una cascata.

In questo approccio, ogni fase deve essere completata e approvata prima di poter passare a quella successiva, rendendo difficile tornare indietro

1.2 Le fasi del modello

Le fasi tipiche sono organizzate in ordine rigoroso



1.2.1 Analisi dei Requisiti

In questa fase iniziale, tutto ciò che il sistema che deve fare viene identificato, documentato e "congelato".

Il risultato è un documento dettagliato di specifica dei requisiti che funge da contratto tra cliente e sviluppatori

1.2.2 Progettazione

Si definisce l'architettura del sistema, le strutture dati, le interfacce e i diagrammi di design.

L'obiettivo è trasformare i requisiti in un piano tecnico solido

1.2.3 implementazione

Gli sviluppatori scrivono il codice.

Il lavoro è focalizzato sulla traduzione del design in software funzionante, spesso lavorando su moduli o unità separate

1.2.4 Collaudo

I componenti vengono integrati e il sistema viene testato nella sua interezza per verificare che rispetti le specifiche definite nella prima fase

1.2.5 Rilascio e Manutenzione

Il sistema viene messo in produzione (distribuito agli utenti).

La fase di manutenzione riguarda la correzione di errori scoperti dopo il rilascio o l'adattamento del sistema a nuovi requisiti ambientali

1.3 Caratteristiche chiave

1.3.1 Plan-driven

L'enfasi è posta sulla pianificazione dettagliata e sulla documentazione.

Il progresso è misurato attraverso il complemento di fasi e artefatti documentali

1.3.2 Specializzazione

Spesso, fasi diverse sono assegnate a team diversi

1.3.3 Rigidità

Una volta che si è superata una fase, non si dovrebbe tornare indietro.

Se emergono problemi, il modello non prevede meccanismi nativi per gestire il cambiamento, se non ricominciando il ciclo da capo

1.4 Vantaggi

1.4.1 Semplicità e chiarezza

È estremamente semplice da gestire.

Le scadenze, i budget e i progressi sono facili da visualizzare su un diagramma di Gantt

1.4.2 Ideale per requisiti stabili

È efficace in progetti dove i requisiti sono ben compresi fin dall'inizio e non cambieranno mai

1.5 Svantaggi

1.5.1 Incapacità di gestire i cambiamenti

Nel mondo reale, i clienti raramente sanno cosa vogliono finché non vedono il software.

Se i requisiti cambiano a metà percorso, i costi di modifica diventano proibitivi

1.5.2 Feedback tardivo

Il cliente vede il software funzionante solo nelle fasi finali del progetto.

Se si accorge che il prodotto non è quello desiderato, il danno economico e temporale è irreparabile

1.5.3 Debito di integrazione

Poichè il test avviene solo alla fine, eventuali errori architetturali o di comunicazione tra i moduli emergono tardi, rendendo la loro correzione estremamente complessa

2. UP - Unified Process

2.1 Cosa è?

Il Processo Unificato è un framework di processo iterativo ed evolutivo ampiamente utilizzato nell'ingegneria del software, specificamente concepito per lo sviluppo di sistemi orientati agli oggetti.

2.2 Le 3 caratteristiche fondamentali dell'UP

Tutto lo sviluppo all'interno del framework UP è guidato da 3 caratteristiche essenziali

2.2.1 Pilotato dai casi d'uso

Lo sviluppo è interamente guidato dalle reali necessità degli utenti finali.

I casi d'uso vengono utilizzati per definire i requisiti, pianificare le iterazioni, guidare la progettazione e strutturare i test, garantendo un coinvolgimento continuo degli utenti per raccogliere feedback

2.2.2 Guidato dai fattori di rischio

Il team non sviluppa le funzionalità in modo casuale o sequenziale, ma affronta e risolve fin dalle primissime iterazioni le problematiche a maggior rischio tecnico o a più alto valore aziendale

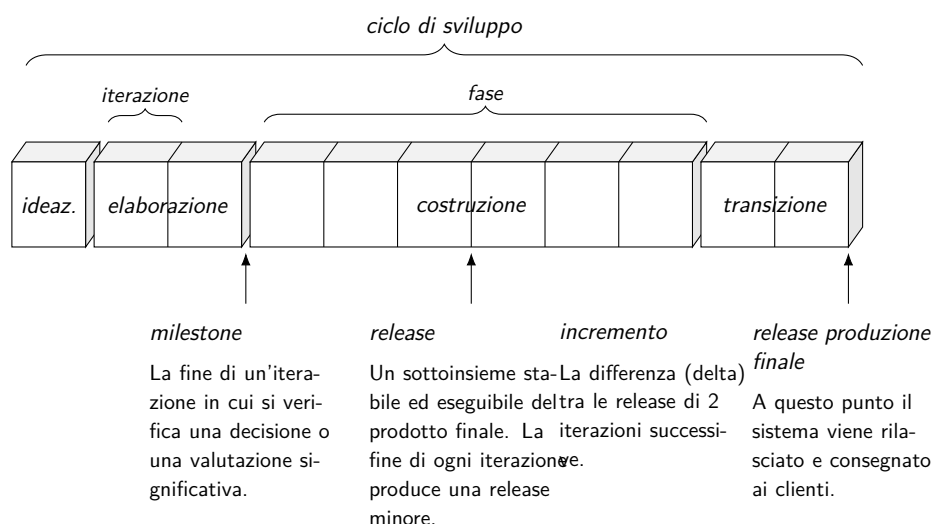
2.2.3 Incentrato sull'architettura

Piuttosto che concentrarsi immediatamente sui dettagli secondari, le fasi iniziali mirano a costruire, testare e stabilizzare un nucleo architettonico solido e coeso

2.3 Le 4 fasi del ciclo di vita

Un progetto gestito con UP organizza il proprio asse temporale suddividendo in 4 macro-fasi sequenziali.

Al termine di ciascuna fase è prevista un "milestone", ovvero un momento in cui si verifica una decisione o valutazione significativa per stabilire se procedere.



2.3.1 Ideazione

Costituisce l'avvio formale del progetto.

L'obiettivo principale non è programmare, ma valutare la fattibilità. In questa fase si definisce una visione approssimativa, si valuta la portata(scope), si individuano i casi d'uso principali e si abbozzano stime preliminari di costi e tempi

2.3.2 Elaborazione

È la fase più critica, incentra sulla realizzazione e sul test iterativo del nucleo dell'architettura.

Qui si mitigano i rischi maggiori, si definisce la maggior parte dei requisiti e si formulano stime molto realistiche su budget e tempistiche

2.3.3 Costruzione

È la fase dedicata alla produzione intensiva.

Vengono sviluppati iterativamente tutti gli elementi rimanenti, che sono tipicamente i più facili e a minor rischio, preparando il sistema per la capacità operative iniziali

2.3.4 Transizione

Ha lo scopo di portare il software agli utenti finali.

Include attività come il beta testing, la correzione degli ultimi difetti e si conclude con la consegna del prodotto finale pronto per la produzione

2.4 Dinamica di Sviluppo: Iterazioni, Discipline e incrementi

Nell'UP, il lavoro non procede in modo lineare con blocchi monolitici di tempo, ma attraverso cicli più agili

2.4.1 Iterazioni

Il tempo è ripartito in iterazioni brevi e con una durata prefissata(timeboxed).

Ogni iterazione è un "mini-progetto" autonomo che include attività(o discipline) di pianificazione, analisi dei requisiti, progettazione, costruzione del codice, integrazione e test

2.4.2 Variazione dell'impegno

Sebbene ogni iterazione possa comprendere tutte le discipline, l'enfasi e l'impegno relativo cambiano a seconda della fase in cui ci si trova.

2.4.3 Release e incremento

Ogni iterazione termina con la produzione di una release, ovvero un sottoinsieme stabile ed eseguibile del prodotto finale.

La differenza funzionale tra due release successive prende il nome di incremento.

Questo processo permette al sistema di evolvere e convergere gradualmente verso la soddisfazione dei requisiti corretti

2.5 Vantaggi

Garantisce un'estrema flessibilità, consentendo di adattare il processo eliminando la burocrazia.

Assicura la mitigazione precoce dei rischi strutturali durante l'elaborazione e, producendo software eseguibile a ogni iterazione, permette di ottenere progressi tangibili e un feedback continuo dagli utenti

2.6 Svantaggi

Per i manager legati a visioni tradizionali, il focus sul software funzionante rispetto alla documentazione massiva può trasmettere un senso di scarsa visibilità burocratica.

Inoltre, crescendo per continui incrementi, la struttura del software tende a degradarsi nel tempo (Debito Tecnico), richiedendo investimenti costanti in refactoring.

Infine esiste il grave rischio di "cattiva interpretazione": se si cerca di definire tutti i requisiti e l'architettura in dettaglio prima di programmare, l'UP fallisce, trasformandosi in un finto e rigido modello a cascata

3. Agile

3.1 Cosa è?

Agile non è un singolo processo rigido, ma piuttosto un termine ombrello che indica una forma di sviluppo iterativo orientata a incoraggiare una risposta rapida e flessibile ai cambiamenti.

All'interno di questa famiglia, il framework più famoso e utilizzato per organizzare il lavoro è Scrum, che ha lo scopo di rilasciare software con il più alto valore nel minor tempo possibile

3.2 Valori e Principi

Agile si fonda su un manifesto valoriale nato nel 2001.

I quattro valori fondamentali affermano che, pur riconoscendo l'importanza degli elementi a destra, si attribuisce più valore a quelli a sinistra:

- Gli individui e le interazioni contano più dei processi e degli strumenti
- Il software funzionante è più importante della documentazione esaustiva
- La collaborazione con il cliente supera la negoziazione dei contratti
- Rispondere al cambiamento è più vitale che seguire perdissequamente un piano

3.3 Flusso di valore: Sprint e Artefatti

La pianificazione Agile è puramente incrementale.

Si basa su cicli di vita brevi in cui si alternano continuamente specifica, sviluppo e validazione

3.3.1 Product Backlog

È il "serbatoio" di tutto il lavoro.

Consiste in un elenco delle cose da fare:

- funzionalità
- requisiti
- storie utente
- task tecnici

3.3.2 Spint (iterazione)

È il cuore pulsante del processo.

Si tratta un'iterazione di sviluppo con una durata prefissata (timeboxing), solitamente compresa tra le 2 e le 4 settimane

3.3.3 Daily Scrum

Un incontro quotidiano in cui il team esamina i progressi fatti nelle ultime 24 ore e definisce le priorità della giornata in corso

3.3.4 Incremento Potenzialmente Rilasciabile

Alla fine di ogni Sprint, il team deve fornire un incremento di software finito,

L'idea è che debba essere pronto per la produzione, senza richiedere ulteriori cicli di test esterni

3.3.5 Velocity

È una metrica fondamentale che stima quanto lavoro il team riesce a smaltire in un singolo Sprint.

Serve per calibrare i carichi di lavoro futuri in base alle reali prestazioni.

3.4 Vantaggi

La pianificazione incrementale rende estremamente facile modificare il processo per riflettere le mutevoli esigenze dei clienti, un requisito fondamentale poichè pochi sistemi aziendali hanno requisiti stabili.

Le consenze continue promuovono la visibilità, la motivazione del team e la soddisfazione del cliente

3.5 Svantaggi

L'assunto che l'incremento di fine Sprint sia sempre "potenzialmente rilasciabile" in pratica non è sempre realizzabile.

Inoltre, richiedendo team piccoli e auto-organizzati, scala con molta difficoltà su progetti enormi in cui un sistema viene sviluppato in siti distribuiti

4. SCRUM

4.1 Cosa è?

Mentre Agile è la "filosofia" di base, Scrum è l'implementazione pratica: un framework iterativo e incrementale progettato per sviluppare e rilasciare prodotti complessi massimizzando il valore nel minor tempo possibile.

4.2 Ruoli in Scrum

A differenza dei modelli tradizionali che prevedono decine di figure gerarchiche, Scrum semplifica l'organizzazione in 3 ruoli fondamentali eliminando le figure di project management classiche

4.2.1 Product Owner

È un individuo che rappresenta la voce del business e del cliente.

Il suo compito è identificare le caratteristiche e i requisiti del prodotto, assegnare loro le priorità per lo sviluppo e revisionare continuamente la lista delle cose da fare.

Assicura che il progetto continui a soddisfare le esigenze critiche di business

4.2.2 Team di Sviluppo

Un gruppo auto-organizzato di sviluppatori.

Una regola d'oro di Scrum è che questo team non dovrebbe superare le 7 persone.

Sono responsabili in autonomia dello sviluppo tecnico del software e della stesura dei documenti essenziali del progetto.

Nessuno dice al team come trasformare i requisiti in software funzionante

4.2.3 ScrumMaster

È il "facilitatore" del processo.

Il suo scopo è garantire che il team comprenda e applichi correttamente la teoria e le pratiche di Scrum, rimuovendo gli ostacoli esterni che potrebbe rallentare il Team di Sviluppo

4.3 Elaborati

Scrum riduce al minimo la documentazione burocratica, concentrandosi su 3 "artefatti" principali che garantiscono trasparenza e ispezione

4.3.1 Product Backlog

È l'elenco dinamico e prioritizzato di tutto il lavoro "da fare".

Può contenere definizioni di nuove funzionalità, requisiti software, storie utente, correzioni di bug o descrizioni di compiti supplementari.

È gestito unicamente dal Product Owner

4.3.2 Sprint Backlog

È il sottoinsieme di elementi del Product Backlog che il Team di Sviluppo seleziona per essere completato nell'iterazione corrente, unito al piano tecnico per realizzarli

4.3.3 Incremento Potenzialmente Rilasciabile

È l'output fondamentale alla fine di ogni iterazione.

Significa che il software prodotto deve trovarsi in uno stato "finito" e non dovrebbe richiedere ulteriore lavoro per essere incorporato nel prodotto finale

4.4 Ciclo di vita

Il lavoro in Scrum non procede per lunghe fasi monolitiche, ma attraverso cicli fissi

4.4.1 Sprint

È il cuore di Scrum, un'iterazione temporale prefissata (timebox), solitamente di 2, 3 o 4 settimane. Durante lo Sprint il team lavora ininterrottamente per trasformare parte del Product Backlog in un Incremento Potenzialmente Rilasciabile

4.4.2 Velocity

È una metrica fondamentale usata in Scrum per misurare quanto lavoro (es. quanti "punti" o requisiti) il team riesce effettivamente a smaltire in un singolo Sprint. Non serve per mettere pressione, ma per calibrare e pianificare realisticamente i carichi di lavoro degli Sprint futuri in base alle prestazioni reali

4.4.3 Eventi chiave

All'interno dello Sprint avvengono eventi fissi come:

1. Sprint Planning
2. Daily Scrum un allineamento quotidiano di 15 minuti del team di sviluppo
3. Sprint Review per mostrare l'incremento al cliente
4. Sprint Retrospective per migliorare il processo stesso

4.5 Vantaggi

4.5.1 flessibilità e Adattabilità

La pianificazione incrementale rende estremamente facile modificare il processo per riflettere le mutevoli esigenze dei clienti.

Questo è vitale, poichè pochissimi sistemi aziendali hanno requisiti stabili fin dall'inizio

4.5.2 Motivazione e Visibilità

Le consegne continue promuovono un'alta visibilità sui progressi, aumentando la motivazione del team e la soddisfazione del cliente

4.6 Svantaggi

4.6.1 Il mito dell'incremento "Shippable"

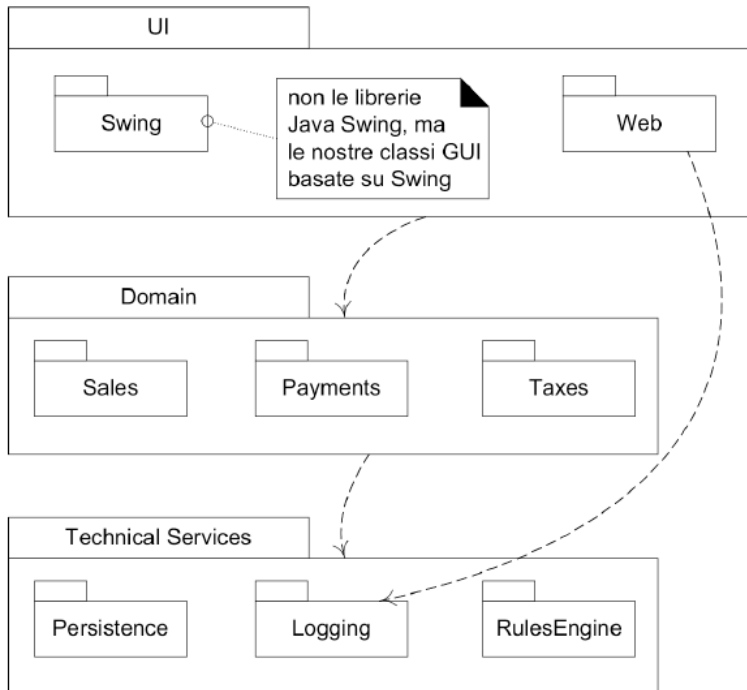
L'assunto che l'incremento di fine Sprint sia sempre pronto per la produzione, senza richiedere ulteriori cicli di test o integrazione di sistema, in pratica non è sempre realizzabile

4.6.2 Problemi di scalabilità

Richiedendo team molto piccoli (max 7 persone), coesi e auto-organizzati, Scrum puro scala con molta difficoltà su progetti enormi in cui lo sviluppo è distribuito in diverse sedi mondiali o richiede centinaia di sviluppatori. In questi casi, i modelli "guidati dal piano" (o framework scalati ibridi) funzionano spesso meglio

5. Architettura Logica

5.1 Cosa è l'Architettura Logica?



- L'architettura logica di un sistema software è l'organizzazione su larga scala delle classi software in package, sottoinsiemi e strati
- Uno stile comune per l'architettura logica è l'architettura a strati

5.2 Architettura a Strati

5.2.1 Cosa è uno strato?

- Uno strato è un gruppo a grana molto grossa di classi, package, o sottosistemi, che ha della responsabilità coese a un aspetto importante il sistema
 - Gli strati più alti ricorrono ai servizi degli strati più bassi (normalmente non avviene il contrario)
- Gli strati di un'applicazione software comprendono normalmente:
 - Presentazione - Interfaccia utente
 - Logica applicativa o strato del dominio - elementi che rappresentano concetti del dominio
 - Servizi tecnici

5.2.2 Architettura a strati stretta

Uno strato può solo richiamare i servizi dello strato immediatamente sottostante

5.2.3 Architettura a strati rilassata

Uno strato più alto può richiamare i servizi di strati più bassi di diversi livelli

5.3 Cosa è un architettura software?

Un architettura software è:

- L'insieme delle decisioni significative sull'organizzazione di un sistema software
- La scelta degli elementi strutturali da cui è composto il sistema e delle relative interfacce
- Insieme al loro comportamento specificato dalla collaborazione tra questi elementi
- La composizione di questi elementi strutturali e comportamentali in sottosistemi via via più ampi
- È lo stile architettonico che guida questa organizzazione

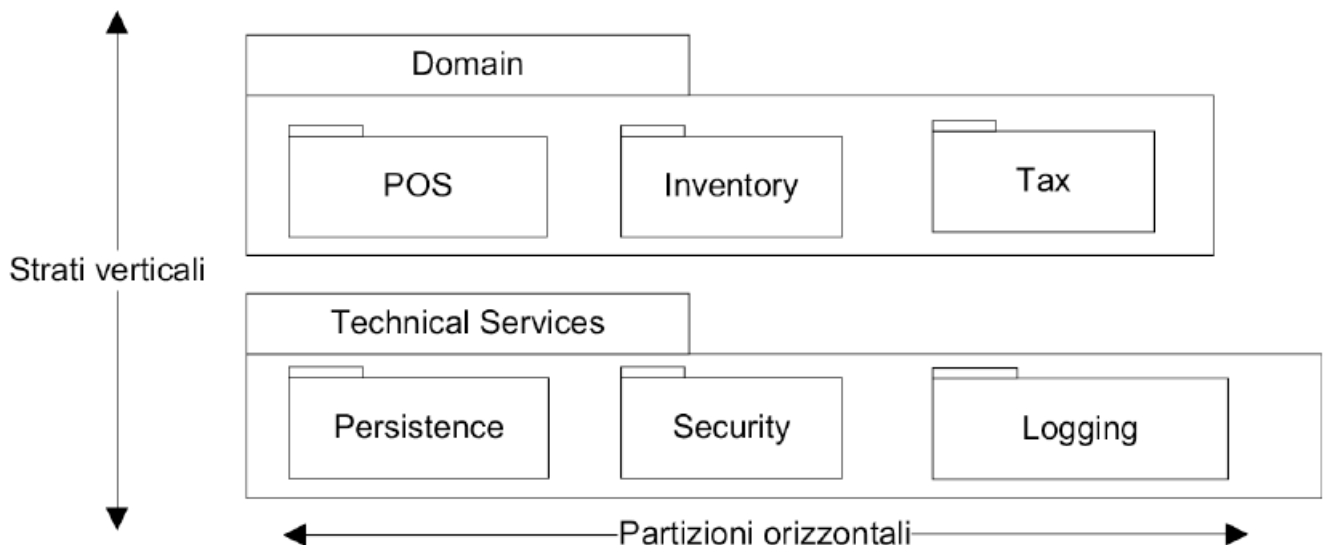
6. Diagramma dei Package

6.1 A cosa serve?

- L'architettura logica può essere illustrata mediante un diagramma dei package
 - Uno strato può essere modellato come un package UML
- Un package UML può raggruppare qualunque cosa (classi, altri package, casi d'uso, ...)
 - È molto comune l'annidamento di package
 - Per mostrare la dipendenza tra i package viene utilizzata una dipendenza UML
 - Un package UML rappresenta un namespace cosicchè è possibile definire due classi con lo stesso nome su package diversi

6.2 Definizioni: Livelli, Strati e Partizioni

- **Livello** : Solitamente indica un nodo fisico di elaborazione
- **Strato** : Una sezione verticale dell'architettura
- **Partizione** : Una divisione orizzontale di sottosistema di uno strato



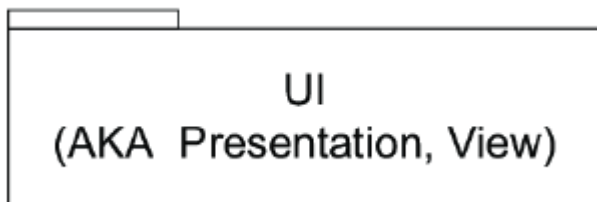
6.3 Progettazione degli strati

6.3.1 Gerarchia degli strati

- Organizza la struttura logica di un sistema in strati separati con responsabilità distinte e correlate, con una separazione netta e coesa degli interessi
 - Gli strati inferiori servizi generali e di basso livello
 - Gli strati superiori sono più specifici per l'applicazione
- Collaborazioni e accoppiamenti vanno dagli strati più alti a quelli più bassi
- L'obiettivo è la suddivisione di un sistema in un insieme di elementi software che, per quanto possibile, possano essere sviluppati e modificati ciascuno indipendentemente dagli altri

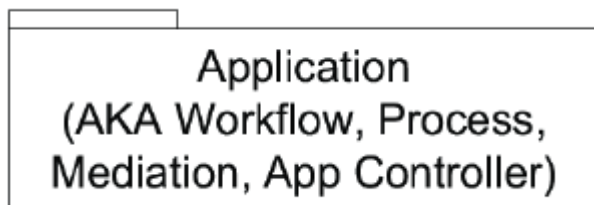
6.3.2 Scelta comune per gli strati

UI



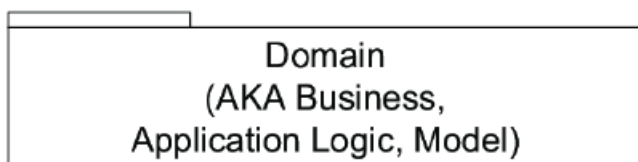
- Finestre della GUI
- Report
- Interfaccia vocale

Application



- Gestisce richieste dello strato UI
- Workflow
- Stato delle sessioni
- Transizioni tra finestre/pagine
- Consolidamento/trasformazione di dati disparati per la presentazione

Domain



- Gestisce richieste dello strato Application
- Implementazione delle regole di dominio
- Servizi di dominio
 - I servizi possono essere utilizzati da una sola applicazione, ma sono possibili anche servizi di interesse per più applicazioni

Business Infrastructure

Business Infrastructure
(AKA Low-level Business Services)

- Servizi aziendali molto generali e di basso livello util

Technical Service

Technical Services
(AKA Technical Infrastructure,
High-level Technical Services)

- Servizi tecnici e framework di livello alto

Foundation

Foundation
(AKA Core Services, Base Services,
Low-level Technical Services/Infrastructure)

- Servizi tecnici e framework di livello basso livello, strutture di dati, thread, matematica

6.3.3 Principio di separazione Modello-Vista

- Gli oggetti di dominio (modello) non devono essere connessi o accoppiati direttamente agli oggetti UI
 - Non permettere a un oggetto di dominio Sale di avere un riferimento a un oggetto finestra JFrame di Java Swing
- Non mettere la logica applicativa nei metodi di un oggetto è l'interfaccia utente
 - Gli oggetti UI dovrebbero solo inizializzare gli elementi dell'interfaccia utente, ricevere eventi UI, e delegare le richieste di logica applicativa agli oggetti non UI

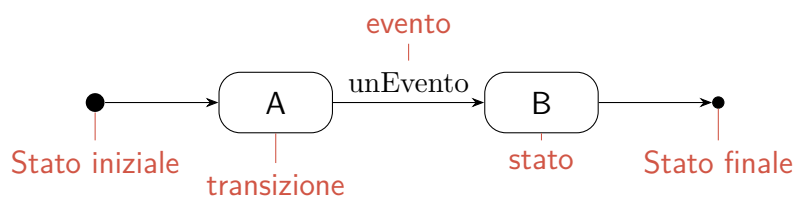
7. Diagramma della Macchina a Stati

7.1 A cosa servono?

I diagrammi di macchina a stati sono strumenti fondamentali utilizzati per modellare il comportamento dinamico di classificatori, come classi, casi d'uso, sottosistemi e interi sistemi.

Sono particolarmente utili per descrivere oggetti "reattivi" o complessi che possiedono un ciclo di vita definito da una precisa successione di stati, transizioni ed eventi

7.2 Sintassi base



- Ogni macchina a stati dovrebbe avere uno stato iniziale che indica il primo stato della sequenza
- A meno che esista un ciclo perpetuo di stati, dovrebbe avere uno stato finale che termina la sequenza di transizioni

7.3 Stati

7.3.1 Definizione

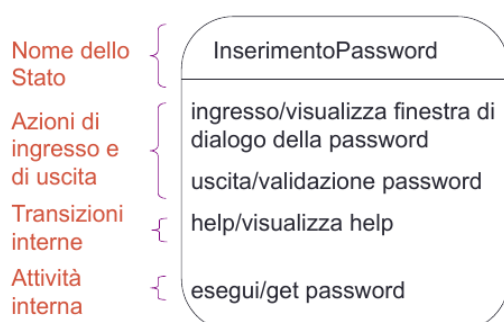
Uno stato rappresenta una condizione o una situazione nella vita di un oggetto, durante la quale quest'ultimo soddisfa una condizione, esegue un'attività o è in attesa di un evento.

Lo stato corrente di un oggetto dipende dai suoi comportamenti passati e definisce come risponderà agli eventi futuri

Lo stato di un oggetto in qualsiasi momento è determinato da:

- I valori dei suoi attributi
- Le relazioni che ha con altri oggetti
- Le attività che sta eseguendo

7.3.2 Sintassi



- Le azioni sono istantanee e non interrompibili
- Le transizioni interne occorrono dentro lo stato.
Non causano la transizione in un nuovo stato
- Le attività richiedono un intervallo di tempo finito e sono interrompibili

7.4 Eventi

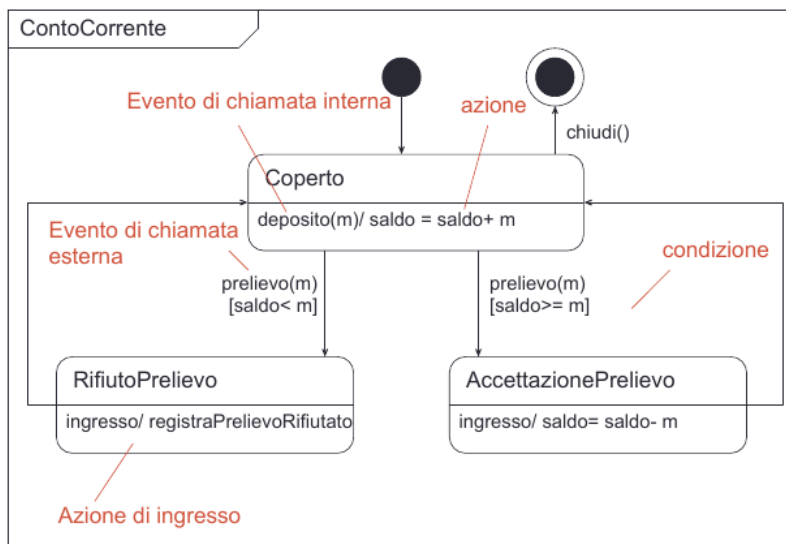
7.4.1 Definizione

Un evento è la specifica di un'occorrenza di interesse che ha una collocazione nel tempo e nello spazio

Gli eventi attivano le transizioni e possono essere di 4 tipi

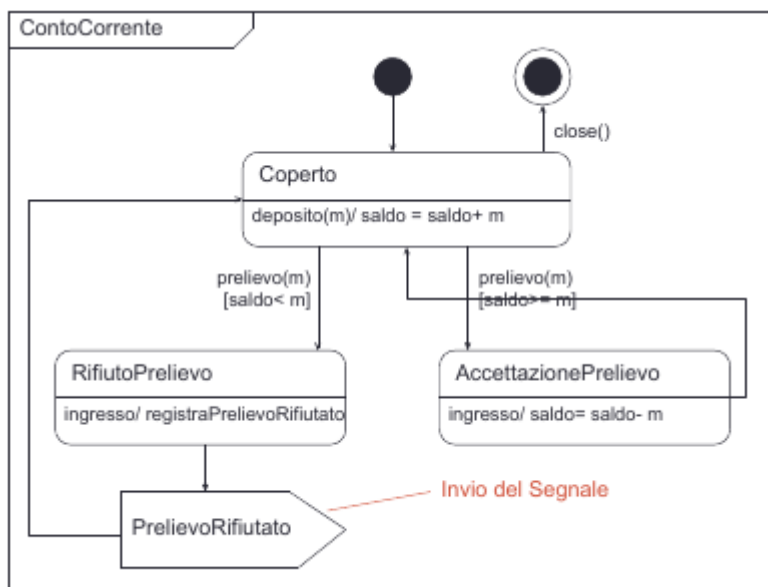
7.4.2 Tipologie di eventi

Eventi di Chiamata



- Rappresentano la chiamata a una specifica operazione.
- Affinchè siano validi, l'evento dovrebbe avere la stessa segnatura di un'operazione della classe di contesto in cui si trova la macchina a stati

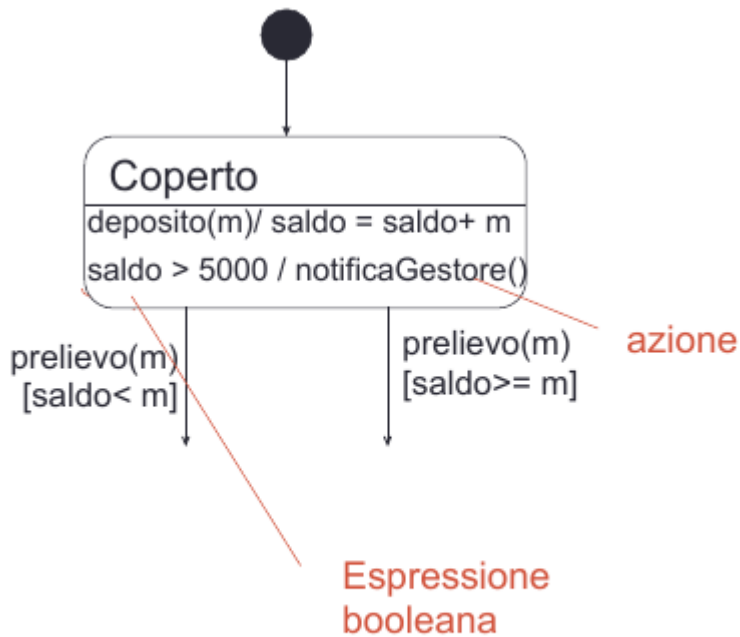
Eventi di Segnale



- Un segnale è un pacchetto di informazioni inviato in modo asincrono tra oggetti
- I suoi attributi contengono le informazioni da trasportare e, graficamente, la sua ricezione è indicata da un pentagono concavo

«segnale» PrelievoRifiutato
data: Date numeroConto: long saldoDisponibile: double

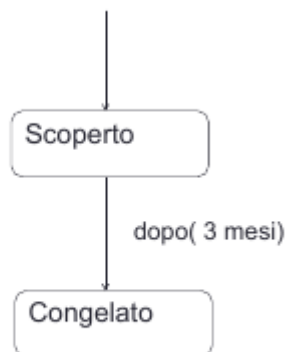
Eventi di Variazione



- Sono rappresentati da un'espressione booleana continua.
- La transizione scatta nell'esatto momento in cui il valore dell'espressione passa da falso a vero
- Implementativamente, questo richiede un ciclo di test continuo per tutta la durata dello stato

Contesto: classe ContoCorrente

Eventi Temporal



Contesto: classe ContoCredito

- Gli eventi temporali occorrono quando un'espressione di tempo diventa vera
- Le parole chiavi sono, dopo e quando

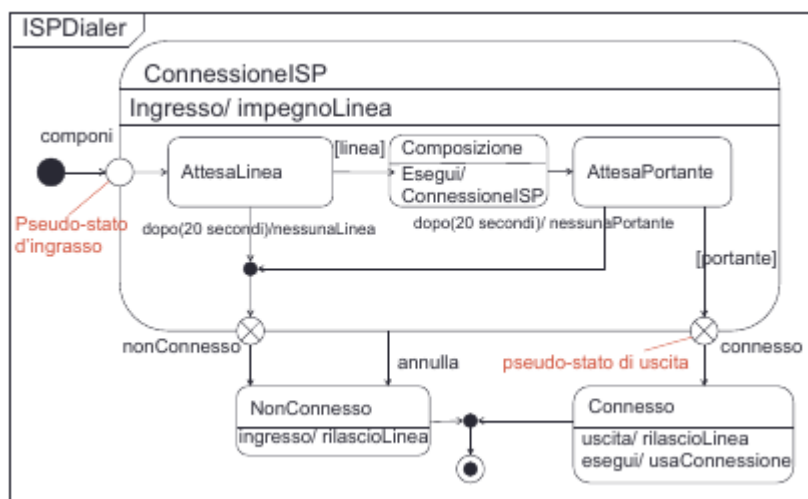
7.5 Stati Composti

7.5.1 Definizione

Uno stato composto è uno stato che contiene al suo interno una o sotto-macchine a stati (regioni).

Vengono utilizzati per organizzare e semplificare sistemi complessi, evitando la proliferazione di transizioni ripetitive e introducendo concetti avanzati come l'ereditarietà delle transizioni, la scomposizione gerarchica e la concorrenza.

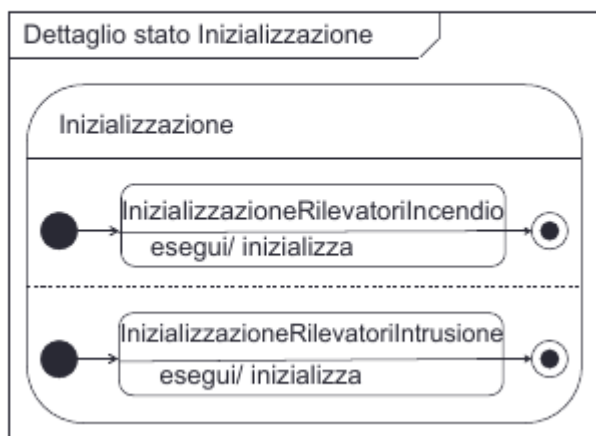
7.5.2 Stati Composti Semplici (Non-Ortogonal)



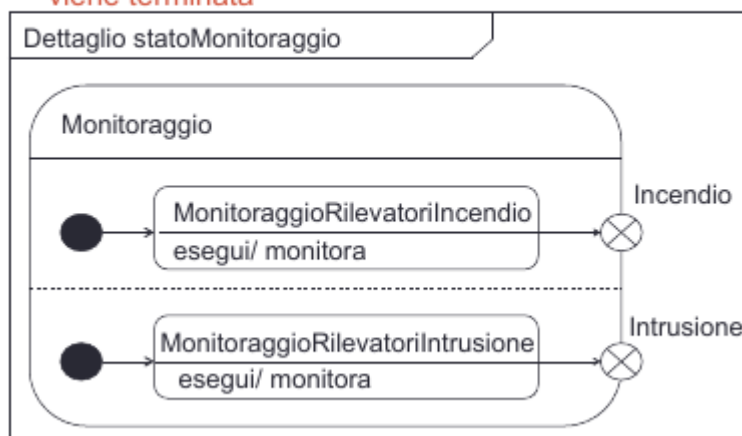
- Contengono una sola regione di sotto-stati.
- Quando l'oggetto si trova nel super-stato, può essere attivo uno e un solo sotto-stato interno alla volta.

7.5.3 Stati Composti Ortogonali

Uscita sincronizzata – si esce dallo stato composto quando tutte le regioni sono terminate



Uscita asincrona - si esce dallo stato composto quando una regione termina. L'altra sotto-macchina viene terminata



- Contengono due o più regioni parallele (separate graficamente da una linea tratteggiata)
- Quando si entra in uno stato ortogonale, l'oggetto si trova contemporaneamente in un sotto-stato di ciascuna regione.
- Quando serve modellare attività concorrenti o indipendenti dello stesso oggetto

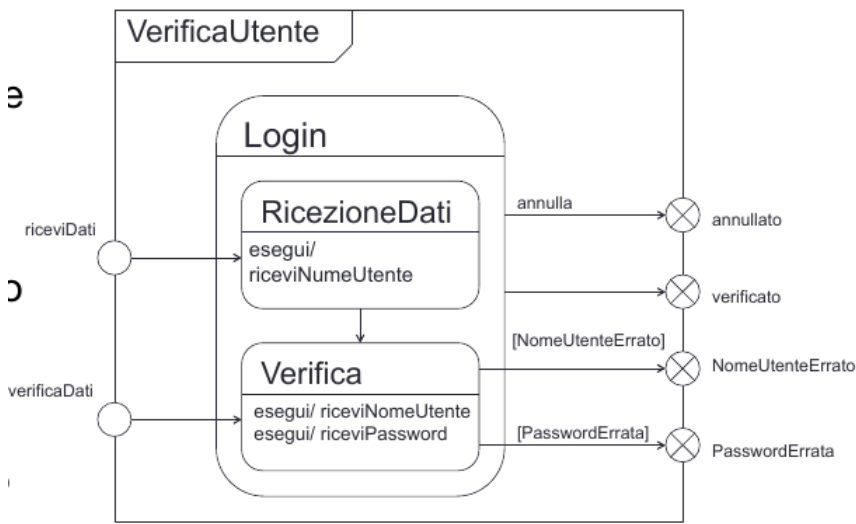
7.6 Sottomacchine

7.6.1 Cosa sono?

Come visto in precedenza, uno stato composito può contenere al suo interno una o più "sotto-macchine" annidate (suddivise in regioni).

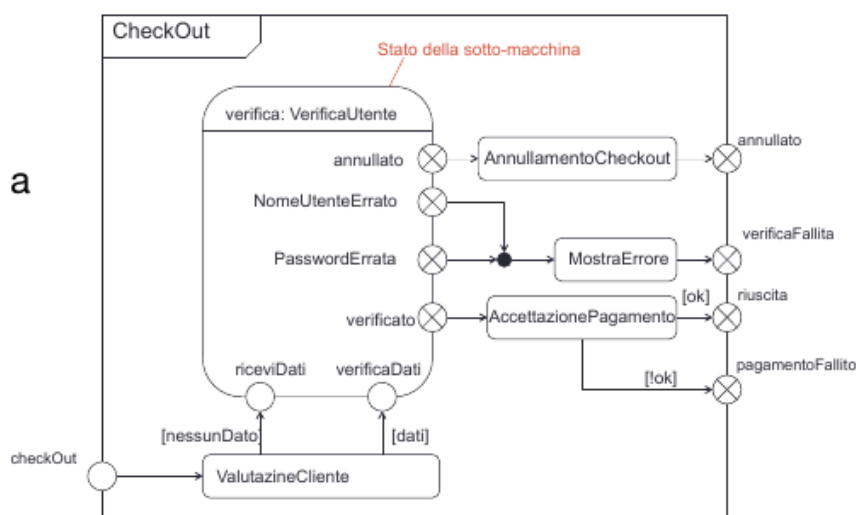
Tuttavia, modellare sistemi complessi disegnando tutte le sottomacchine all'interno di un unico grande diagramma lo renderebbe rapidamente enorme e illeggibile

7.6.2 Stati della Sotto Macchina



- Se vogliamo fare riferimento a questa macchina a stati in altre macchine a stati, senza ingombrare i diagrammi, dobbiamo usare gli stati della sotto-macchine
- Gli stati della sotto-macchina fanno riferimento a un'altra macchina a stati
- Gli stati della sotto-macchina sono semanticamente equivalenti agli stati composti

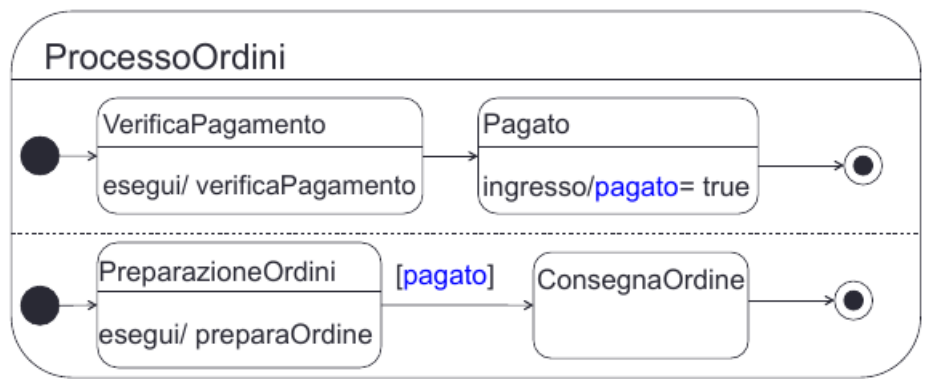
7.6.3 Sintassi degli stati della Sotto-Macchine



- Uno stato della sotto-macchina è equivalente a includere una copia della sotto-macchina al posto dello stato della sotto-macchina

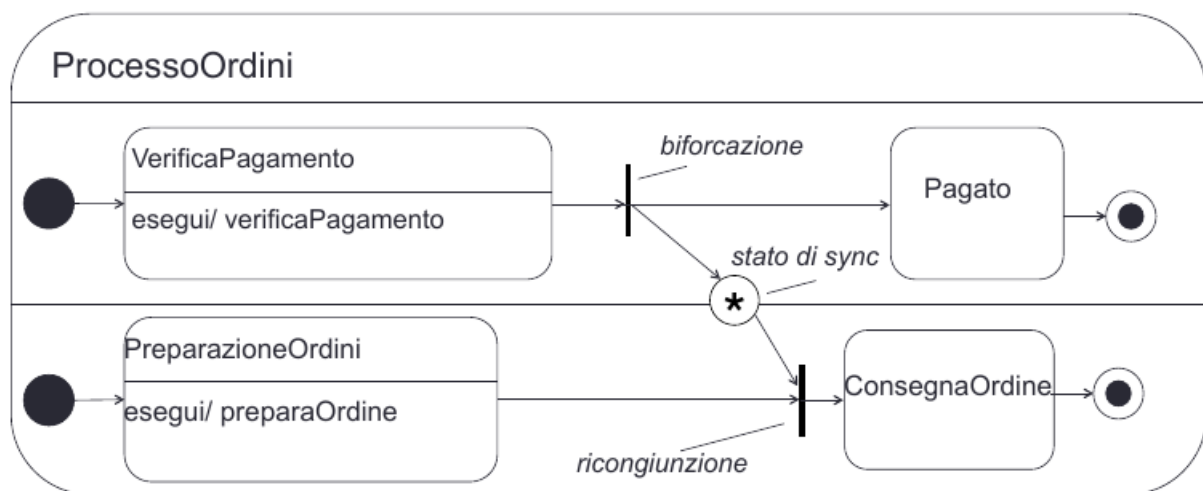
7.6.4 Comunicazione tra sotto-macchine

La comunicazione avviene utilizzando l'attributo ***pagato*** come flag: La sotto-macchina in alto setta il flag e la sotto-macchina in basso usa questo come condizione di guardia



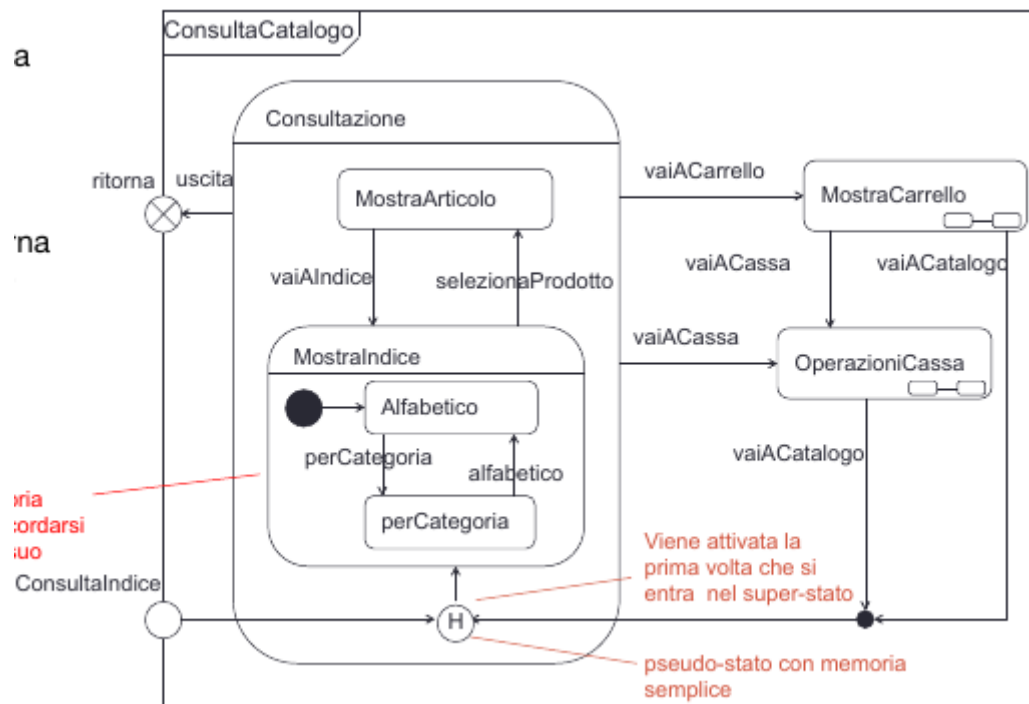
- Capita spesso di avere l'esigenza di far comunicare due sotto-macchine
- Biforcazioni e ricongiunzioni possono essere utilizzate per creare sotto-macchine concorrenti e per risincronizzare
- La comunicazione asincrona è ottenuta da una sotto-macchina configurando un flag per un'altra sotto-macchina

7.6.5 Comunicazione tramite stato di sync



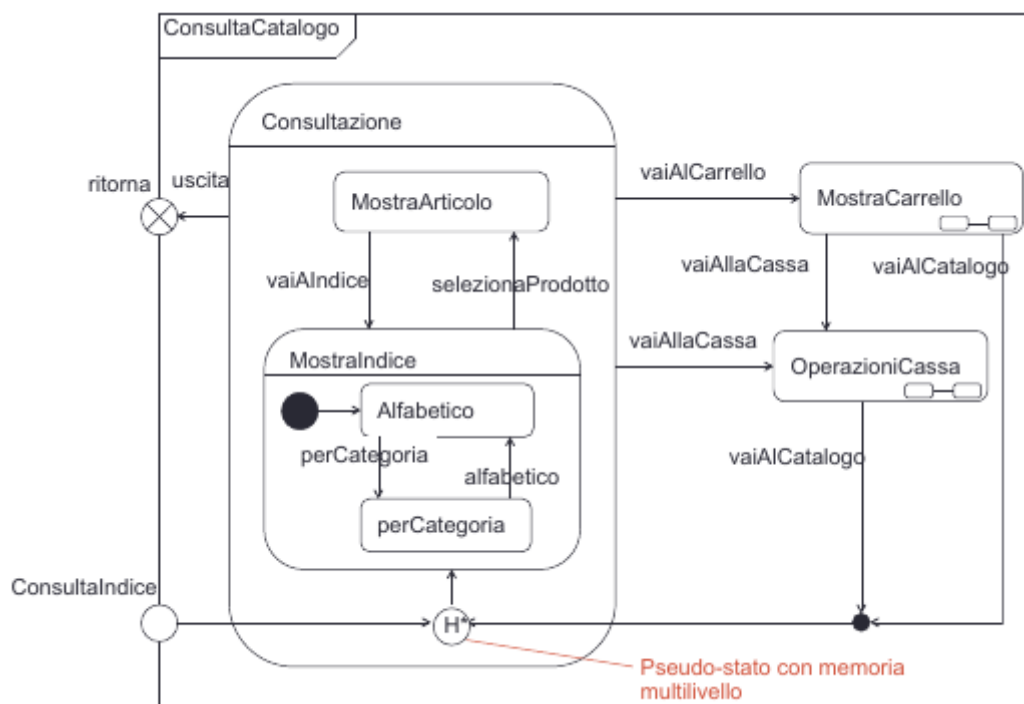
- Lo stato di sync è uno stato speciale il cui compito è quello di tenere traccia di ogni singola attivazione della sua unica transizione di input
- Lo stato di sync è come una coda: si aggiunge un elemento alla coda ogni volta che viene attivata la transizione di input
 - numero limitato di elementi specificando all'interno dello stato di sync un numero intero positivo
 - Asterisco per un numero illimitato di elementi

7.6.6 Stati con memoria semplice



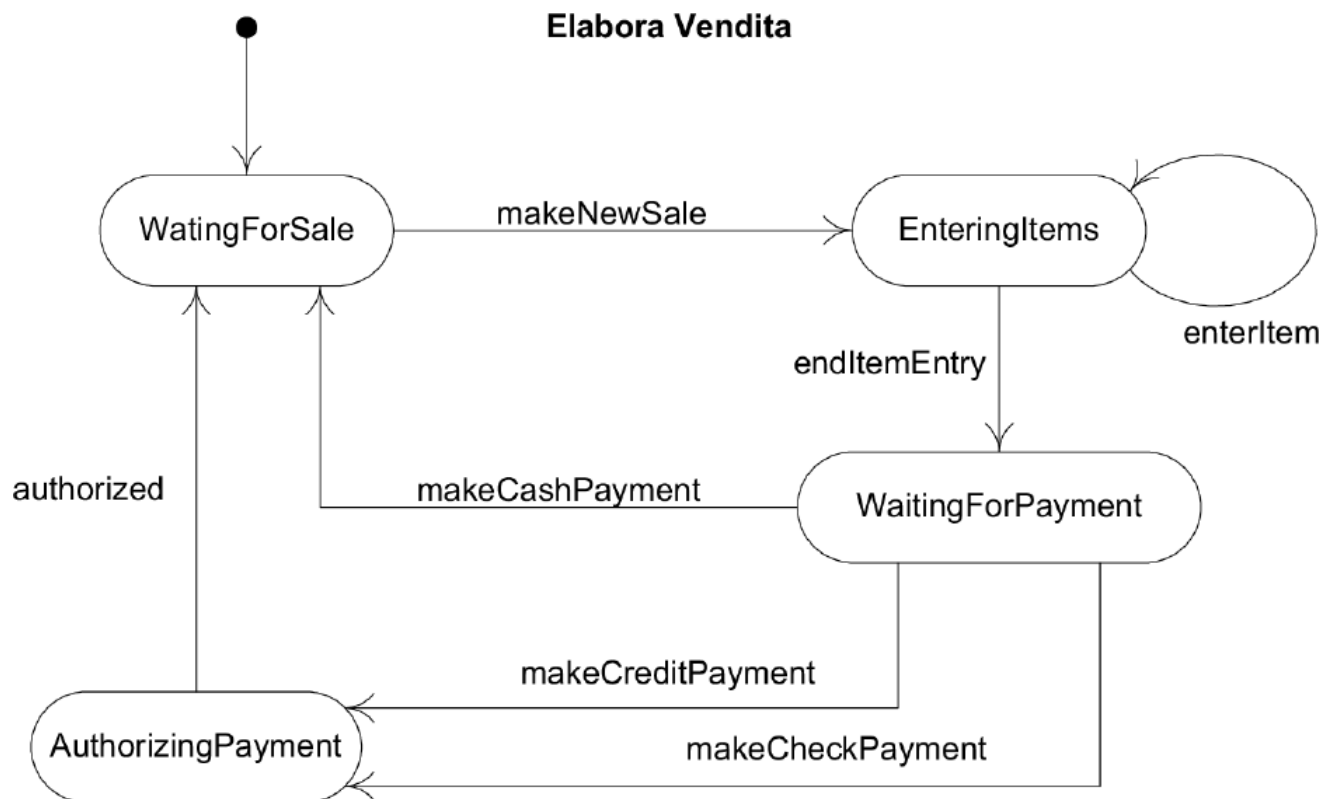
- Lo pseudo-stato con memoria semplice ricorda in quale sottostato si era quando si è lasciato il super-stato
- In seguito quando si ritorna da uno stato esterno allo stato con memoria, l'indicatore ridirezione la transizione sull'ultimo sottostato memorizzato

7.6.7 Sottostato con memoria multilivello



Uno stato con memoria multilivello può non solo ricordarsi l'ultimo sottostato attivo allo stesso livello dello pseudo-stato ma anche l'eventuale ultimo sottostato attivo di questo stato attivo e così via

7.7 Esempio: Elabora vendita



8. Pattern GRASP

8.1 Definizione

I pattern GRASP (General Responsibility Assignment Software Pattern) sono un insieme di principi fondamentali e linee guida utilizzati nella **progettazione del software orientata agli oggetti** per supportare il progettista nell'assegnazione delle responsabilità a classi e oggetti.

8.1.1 Approccio RDD

Più che schemi di codice preconfezionati, i GRASP deniscano un approccio metodologico noto come **Responsability-Driven Design (Progettazione Guidata alla responsabilità)** formalizzando sotto forma di pattern le regole e le soluzioni idiomatiche applicate.

8.1.2 Principi fondamentali

- **Focus sulle Responsabilità:**

Nella progettazione a oggetti, i requisiti espressi nei casi d'uso vengono soddisfatti traducendoli in responsabilità per le classi del software.

Tali responsabilità si dividono in 2 macro-categorie:

- **Responsabilità di FARE (Doing):** Come eseguire un'azione di business, creare un altro oggetto o coordinare attività.
- **Responsabilità di CONOSCERE (Knowing):** Come la conoscenza dei propri dati privati incapsulati, delle relazioni o di elementi calcolabili.

- **Applicazione Dinamica:**

Lo strumento principale in cui si applicano e si concretizzano i pattern GRASP è il disegno dei **diagrammi di interazione UML**.

È proprio nel momento in cui si definiscono i messaggi scambiati tra gli oggetti che il progettista sceglie a quale entità assegnare una specifica funzione.

- **Obiettivo Qualificativo:**

L'applicazione metodica di questi pattern mira a massimizzare metriche software essenziali, riducendo l'impatto dei cambiamenti futuri e migliorando la manutenibilità.

I due capisaldi di questo obiettivo sono il raggiungimento di un **accoppiamento basso**(Low Coupling) e di una **coesione alta**(High Cohesion)

8.1.3 Relazione tra gli elaborati

8.2 Rappresentazione di un Pattern GRASP

Nome del pattern:	Information Expert
Problema:	Qual è un principio di base con cui assegnare responsabilità agli oggetti?
Soluzione:	Assegna una responsabilità alla classe che ha le informazioni necessarie per soddisfarla.

9. Pattern GRASP di base

9.1 Creator

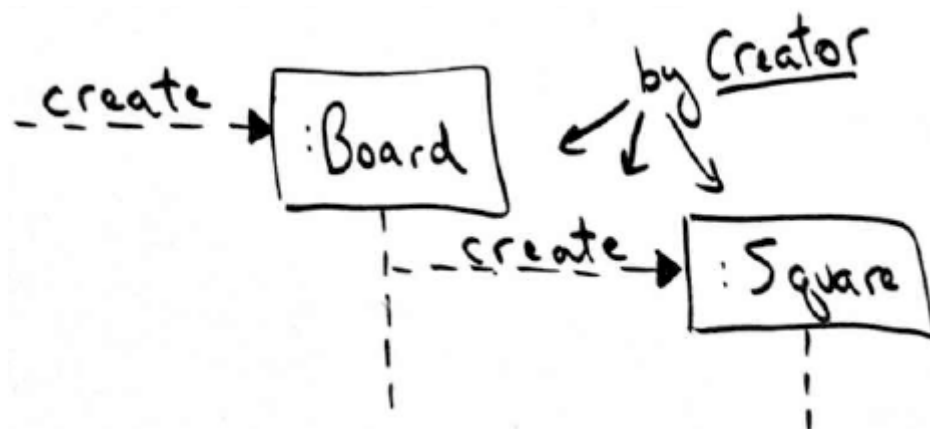
9.1.1 Definizione

Nome:	Creator
Problema:	Chi crea un oggetto A?
Soluzione :	Assegna alla classe B la responsabilità di creare un'istanza della classe A se una delle seguenti condizioni è vera: • B "contiene" o aggrega una composizione oggetti di tipo A • B registra A • B utilizza strettamente A • B possiede i dati per l'inizializzazione di A

9.1.2 Esempio diagramma delle classi



9.1.3 Esempio diagramma di sequenza di stato

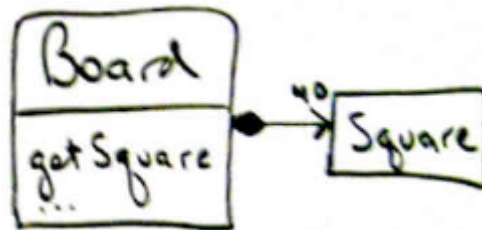


9.2 Information Expert

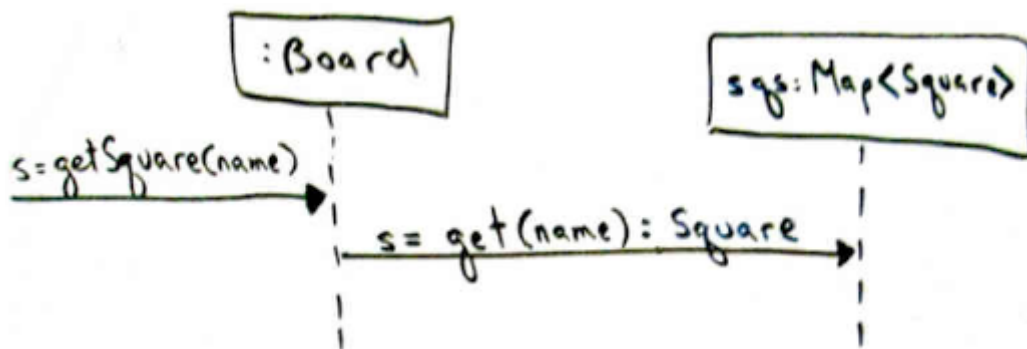
9.2.1 Definizione

Nome:	Information Expert
Problema:	Qual'è un principio di base per assegnare responsabilità agli oggetti
Soluzione :	Assegna una responsabilità alla classe che possiede le informazione necessarie per soddisfarla

9.2.2 Esempio diagramma delle classi



9.2.3 Esempio diagramma di sequenza di stato



9.3 Low Coupling

9.3.1 Definizione

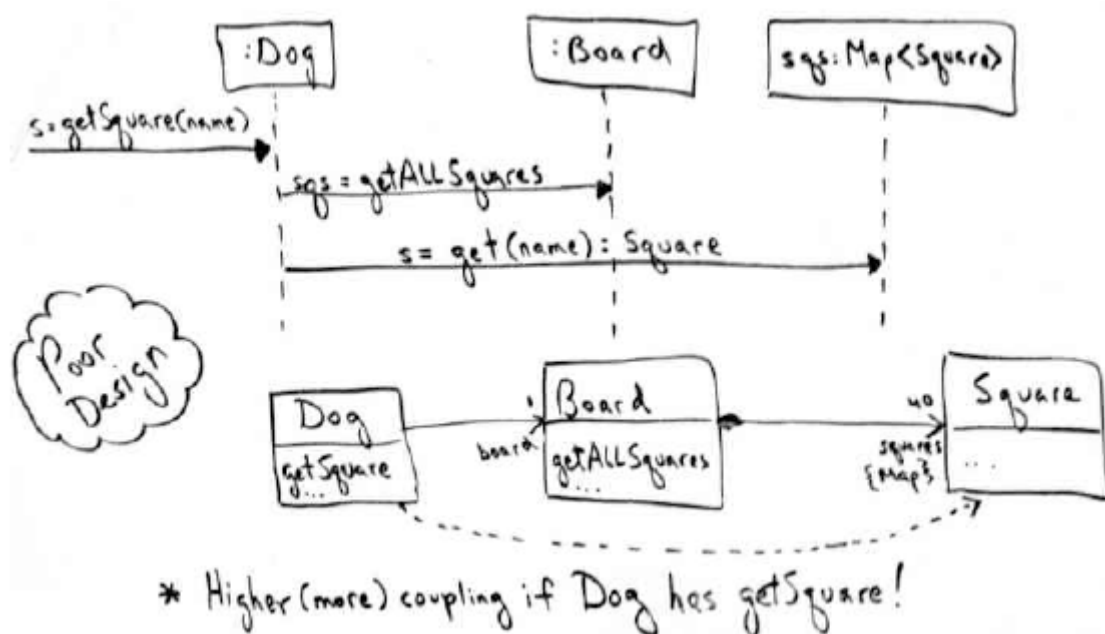
Nome:	Low Coupling
Problema:	Come ridurre l'impatto dei cambiamenti?
Soluzione :	Assegna le responsabilità in modo tale che l'accoppiamento (non necessario) rimanga basso. Usa questo principio per valutare le alternative

9.3.2 Effetto di Information Expert

Information Expert guida verso una scelta che sostiene Low Coupling

- Expert chiede di trovare l'oggetto che possiede la maggior parte delle informazioni richieste per la responsabilità
- Se ingoriamo i consigli di Information Expert, l'accoppiamento complessivo è peggiore
 - Ci sono più oggetti o informazioni che devono essere condivisi

9.3.3 Esempio di alto accoppiamento

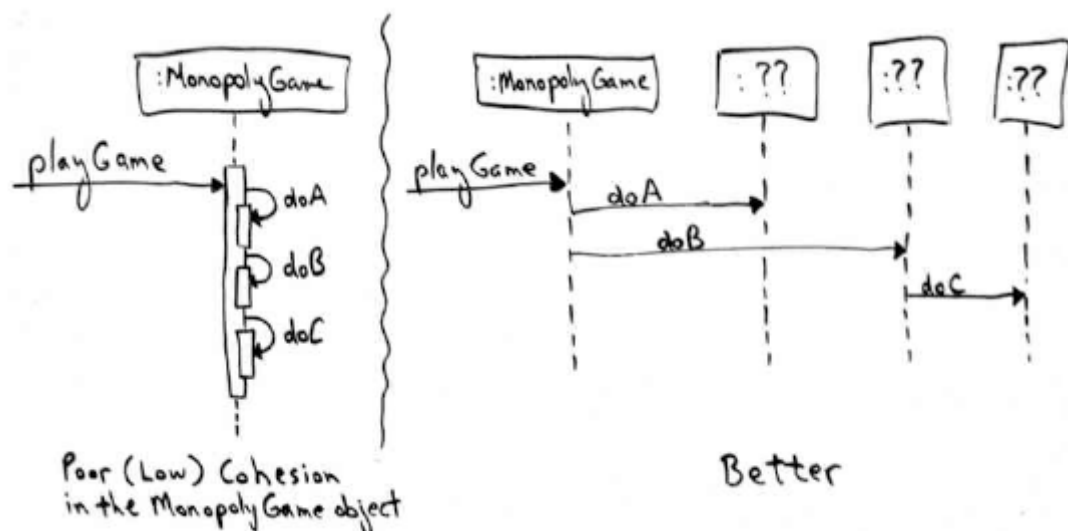


9.4 Controller

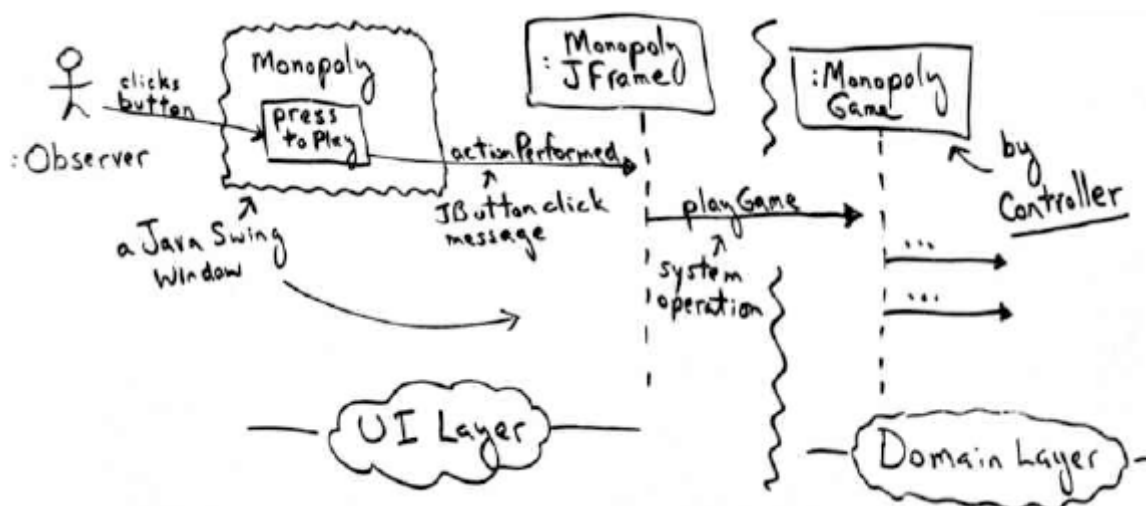
9.4.1 Definizione

Nome:	Controller
Problema:	Qual'è il primo oggetto oltre lo strato UI a ricevere e coordinare un'operazione di sistema
Soluzione :	Assegna la responsabilità a un oggetto che rappresenta una delle seguenti scelte: • Rappresentare il "sistema" complessivo, un "oggetto radice", un dispositivo all'interno del quale viene eseguito il software, un punto d'accesso al software o un sottosistema principale (sono tutte varianti di un facade controller) • Rappresenta uno scenario di un caso d'uso all'interno del quale si verifica l'operazione di sistema (un controller di caso d'uso o controller di session)

9.4.2 Possibili approcci



9.4.3 Esempio MonopoluGame



9.5 High Cohesion

9.5.1 Definizione

Nome:	High Cohesion
Problema:	Come mantenere gli oggetti focalizzati, comprensibili e gestibili e, come effetto collaterale, sostenere Low Coupling?
Soluzione :	Assegna le responsabilità in modo tale che la coesione rimanga alta. Usa questo principio per valutare le alternative

10. Patter GRASP Avanzati

10.1 Pure Fabricator

10.1.1 Definizione

Nome:	Pure Fabricator
Problema:	Quale oggetto deve avere la responsabilità quando non si vogliono violare High Cohesion e Low Coupling, o altri obiettivi, ma le soluzioni suggerite da Expert non sono appropriate?
Soluzione :	Assegnare un insieme di responsabilità altamente coeso a una classe artificiale o di convenienza, che non rappresenta un concetto del dominio del problema, ma piuttosto è una classe inventata per sostenere coesione alta, accoppiamento basso e riuso

10.1.2 Esempio POS NextGen

Vogliamo salvare l'oggetto di tipo Sale in una base di dati razionale.

Solitamente questa responsabilità è assegnata alla classe stessa, ma Sale perderebbe di prestazioni:

- Meno coesione perchè introduciamo le operazioni orientate ai db
- Aumento dell'accoppiamento all'interfaccia del db
- Diminuirebbe il riuso futuro della classe stessa
- Possibile codice duplicato con altre classi



10.2 Polymorphism

10.2.1 Definizione

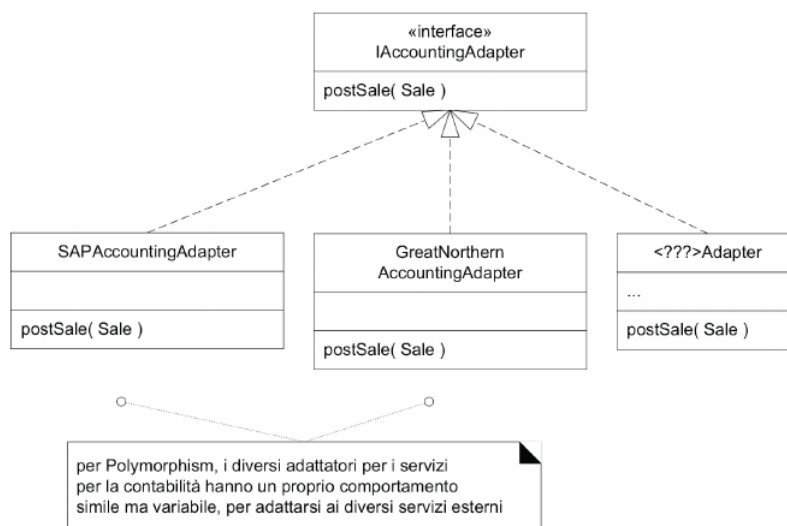
Nome:	Polymorphism
Problema:	Come gestire alternative basate sul tipo? Come creare componenti software inseribili?
Soluzione :	Quando le alternative e comportamenti correlati variano con il tipo, allora assegna la responsabilità del comportamento ai tipi per i quali il comportamento varia, utilizzando operazione polimorfe

10.2.2 Esempio POS NextGen

Vogliamo gestire i diversi sistemi di contabilità di terze parti.

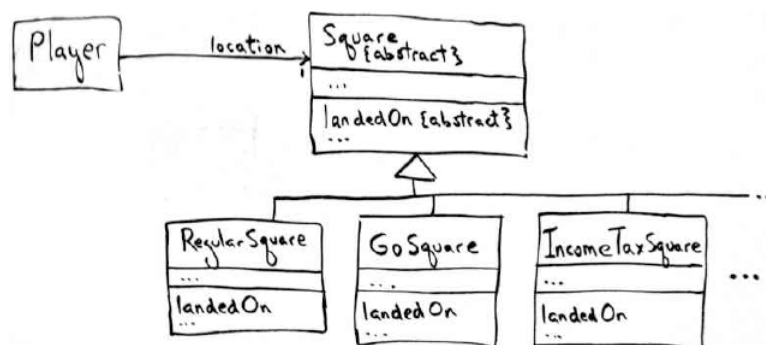
Dobbiamo far sì che il sistema di contabilità di POS NextGen possa comunicare con loro.

Così facendo abbiamo più implementazioni simili ma che si adattano a differenti richieste



10.2.3 Esempio Monopoly

Nel gioco del Monopoli le caselle possono avere un comportamento differente in base al loro tipo, anche se condividono delle caratteristiche comuni



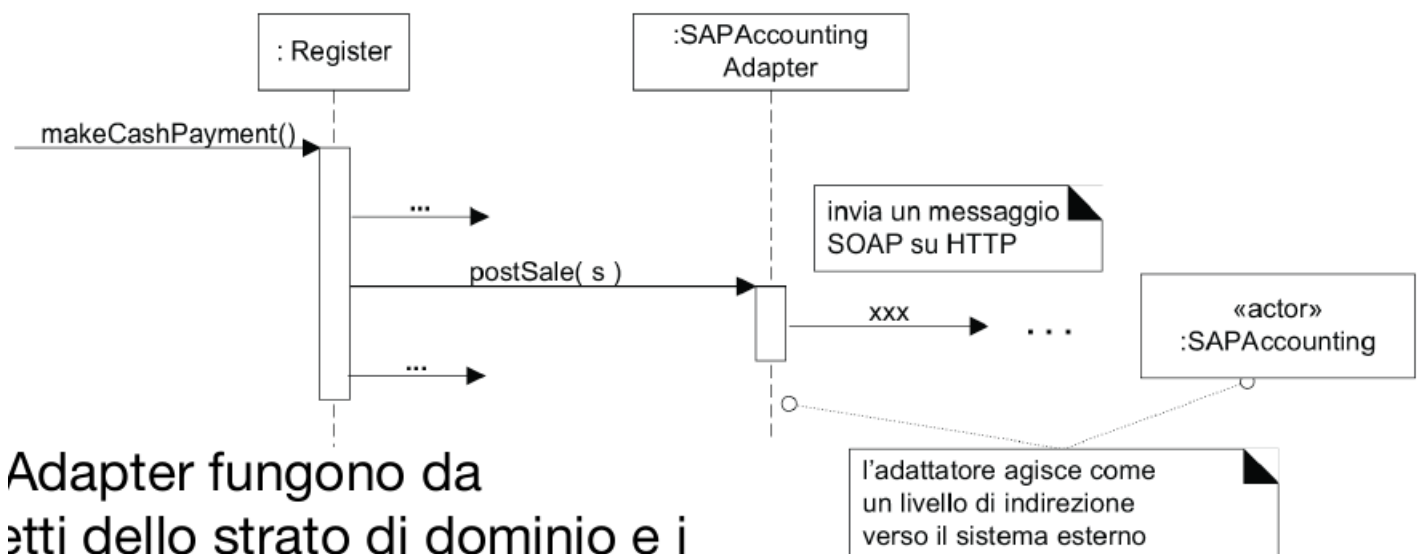
10.3 Indirection

10.3.1 Definizione

Nome:	Indirection
Problema:	Dove assegnare una responsabilità, per evitare l'accoppiamento diretto tra più elementi? Come disaccoppiare gli oggetti in modo da sostenere un alto riuso e un accoppiamento basso?
Soluzione :	Si assegna la responsabilità ad un oggetto intermediario che media tra altri componenti o servizi, in modo che non ci sia un accoppiamento diretto tra di essi • L'intermediario crea una indirezione tra le componenti • Accoppiamento più basso tra componenti

10.3.2 Esempio POS NextGen

- Gli oggetti IAccountingAdapter fungono da intermediari tra gli oggetti dello strato di dominio e i servizi esterni
- Aggiungendo un livello di indicazione e il polimorfismo gli oggetti adattatori proteggono il progetto interno da variazioni nelle interfacce esterne



10.4 Protected Variations

10.4.1 Definizione

Nome:	Protected Variations
Problema:	Come progettare oggetti, sottosistemi e sistemi in modo tale che le variazioni o l'instabilità di questi elementi non si riversino su altri elementi?
Soluzione :	Si identificano i punti in cui sono previste delle variazioni, poi si assegnano le responsabilità per creare una "interfaccia" stabile attorno a questi punti

11. Design Pattern GoF

11.1 Cosa sono?

I design pattern GoF (Gang of Four) sono soluzioni progettuali comuni a problemi di progettazione ricorrenti nello sviluppo di software a oggetti.

In termini pratici, rappresentano un insieme specifico di oggetti e classi a cui vengono date delle responsabilità e che collaborano tra loro per risolvere un determinato problema.

11.2 Tabella

Tabella 11.1: Matrice di classificazione dei Design Pattern (GoF)

		Scopo		
		Creazionale	Strutturale	Comportamentale
Raggio d'azione	Classi	Factory Method	Adapter (class)	Interpreter Template Method
	Oggetti	Abstract Factory Builder Prototype Singleton	Adapter (object) Bridge Composite Decorator Facade Flyweight Proxy	Chain of responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

11.3 Adapter

11.3.1 Definizione

Nome: **Adapter**

Problema: Come gestire interfacce incompatibili, o fornire un'interfaccia stabile a componenti simili ma con interfacce diverse?

Soluzione : Converti l'interfaccia originale di un componente in un'altra interfaccia, attraverso un oggetto adattatore intermedio

11.3.2 Esempio POS NextGen

- Il sistema POS NextGen deve supportare diversi tipi di servizi esterni prodotti da terze parti, compresi i sistemi per la contabilità e per l'inventario, i servizi di autorizzazione ai pagamenti, i servizi per il calcolo delle imposte, tra gli altri
- Ciascuno di essi ha una proprio api, diversa da quelle degli altri, che non può essere modificata

Diagramma delle classi

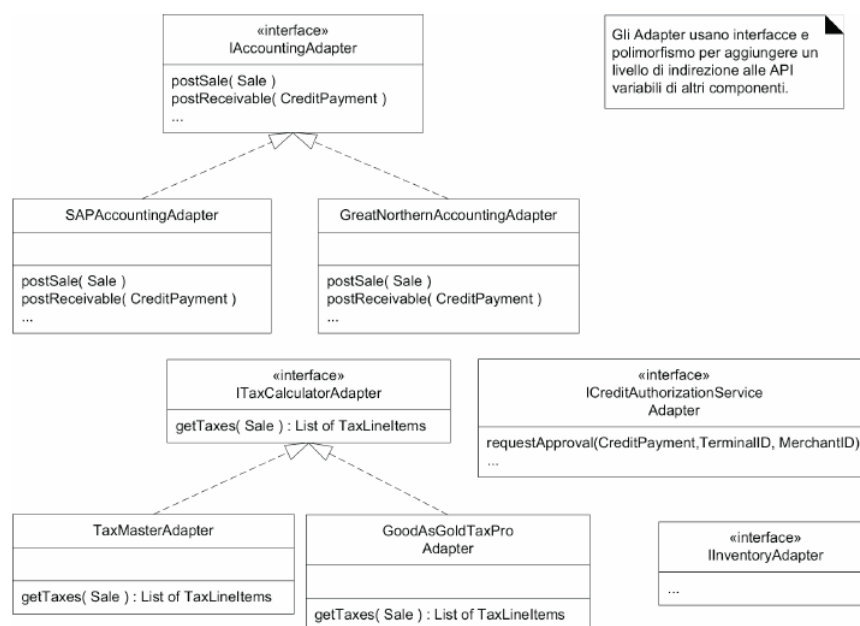
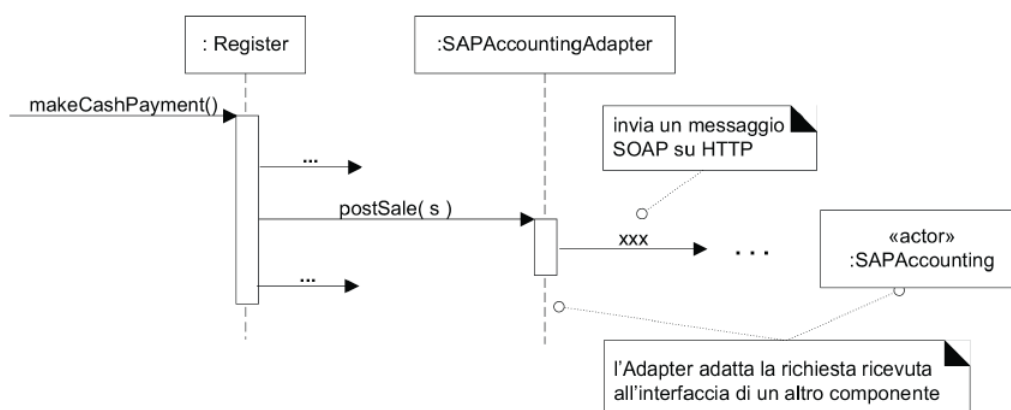


Diagramma di sequenza



11.4 Factory

11.4.1 Definizione

Nome:	Factory
Problema:	Chi deve essere responsabile della creazione di oggetti quando ci sono delle considerazioni speciali, come una logica di creazione complessa, quando si desidera separare la responsabilità di creazione per una coesione migliore, e così via?
Soluzione :	Crea un oggetto Pure Fabrication chiamato una Factory che gestisce la creazione

11.5 Singleton

11.5.1 Definizione

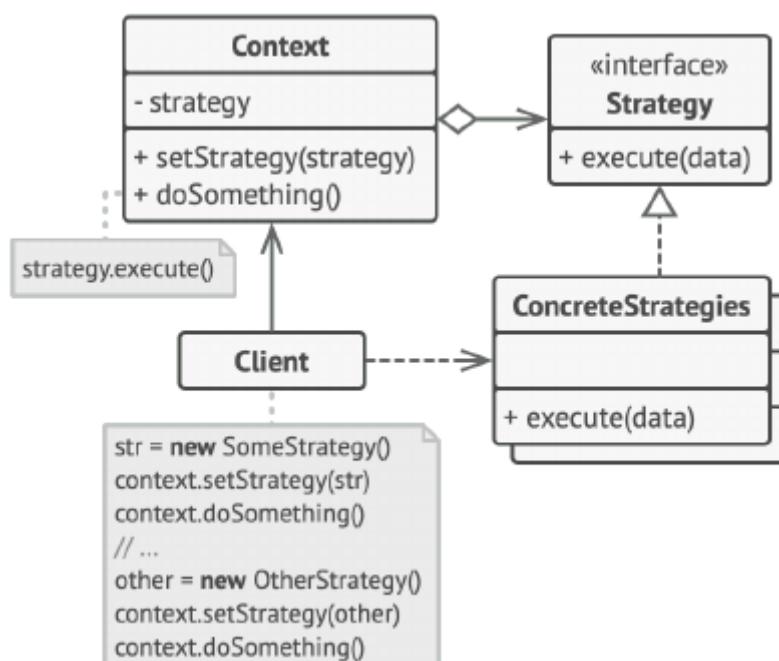
Nome:	Singleton
Problema:	È consentita/richiesta esattamente una sola istanza di una classe, ovvero un "singleton". Gli altri oggetti hanno bisogno di un punto di accesso globale e singolo a questo oggetto
Soluzione :	Definisci un metodo statico (di classe) della classe che restituisce l'oggetto singleton

11.6 Strategy

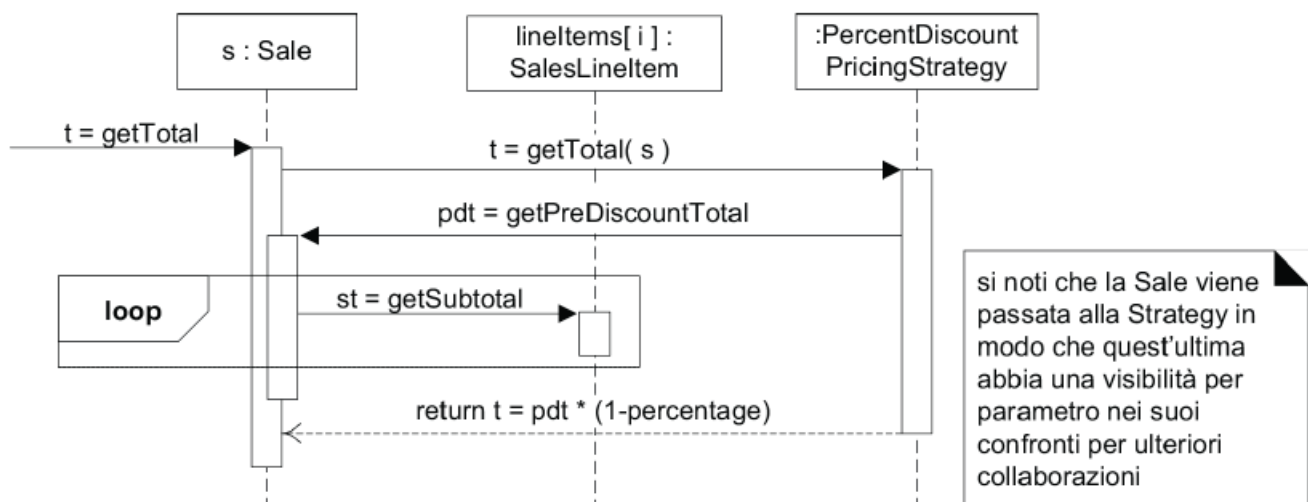
11.6.1 Definizione

Nome:	Strategy
Problema:	Come progettare per gestire un insieme di algoritmi o politiche variabili ma correlati? Come progettare per consentire di modificare questi algoritmi o politiche?
Soluzione :	Definisci ciascun algoritmo/politica/strategia in una classe separata, con un'interfaccia comune

11.6.2 Esempio diagramma delle classi



11.6.3 Esempio diagramma di sequenza



11.7 Composite

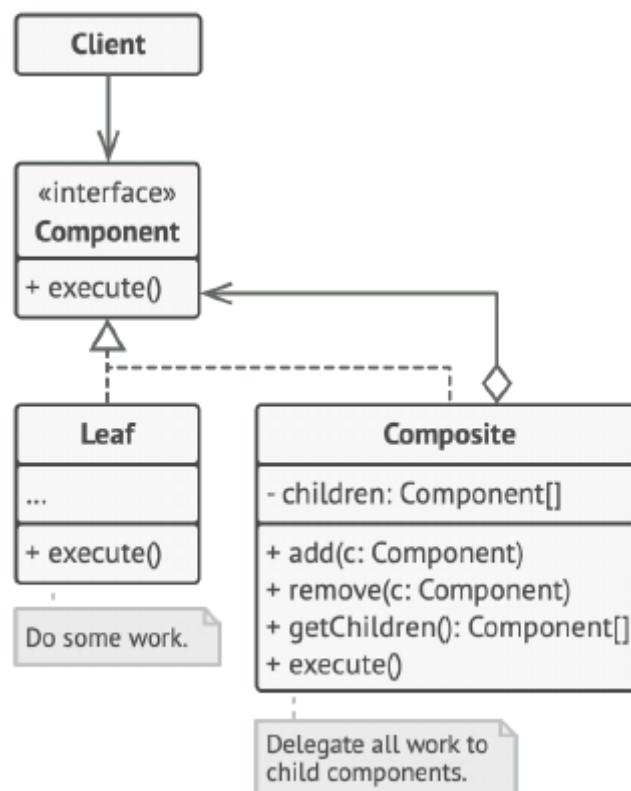
11.7.1 Definizione

Nome:	Composite
-------	------------------

Problema:	Come trattare un gruppo o una struttura composta di oggetti (polimorficamente) dello stesso tipo nello stesso modo di un oggetto non composto (atomico)?
-----------	--

Soluzione :	Definisci le classi per gli oggetti composti e atomici in modo che implementino la stessa interfaccia
-------------	---

11.7.2 Diagramma delle classi



11.8 Facade

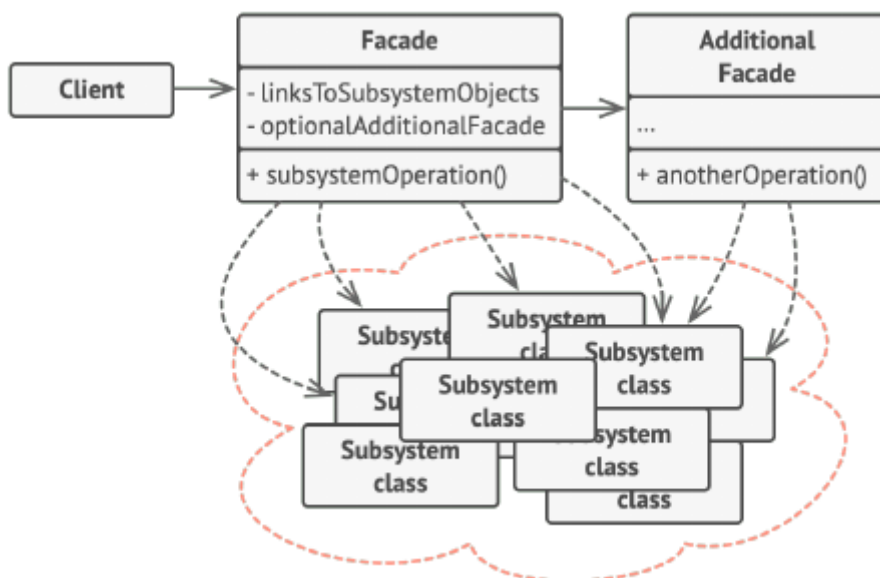
11.8.1 Definizione

Nome: **Facade**

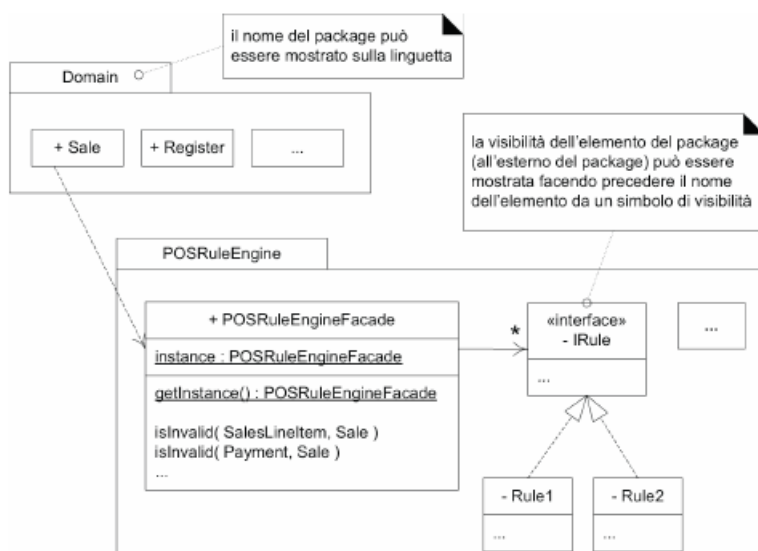
Problema: È richiesta un'interfaccia comune e unificata per un insieme disparato di implementazioni o interfacce, come per definire un sottosistema.
Può verificarsi un accoppiamento indesiderato a molti oggetti nel sottosistema, oppure l'implementazione del sottosistema può cambiare.

Soluzione : Definisci un punto di contatto singolo con il sottosistema, ovvero un oggetto facade che copre il sottosistema.
Questa oggetto facade presenta un'interfaccia singola e unificata ed è responsabile della collaborazione con i componenti del sottosistema

11.8.2 Diagramma delle classi



11.8.3 Diagramma dei Package

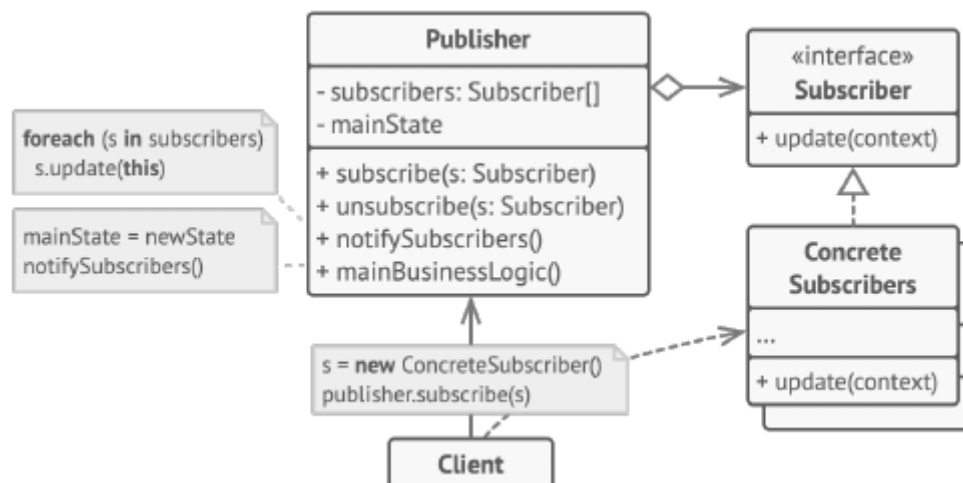


11.9 Observer

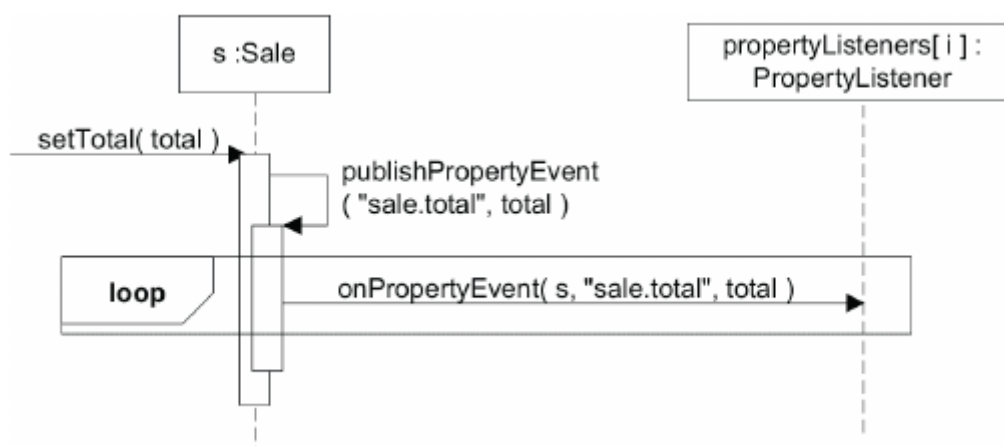
11.9.1 Definizione

Nome:	Observer
Problema:	Diversi tipi di oggetti subscriber sono interessati ai cambiamenti di stato o agli eventi di un oggetto publisher. Ciascun subscriber vuole reagire in un suo modo proprio quando il publisher genera un evento. Inoltre, il publisher vuole mantenere un accoppiamento basso verso i subscriber
Soluzione :	Definisci un'interfaccia subscriber o listener. Gli oggetti subscriber implementano questa interfaccia. Il publisher registra dinamicamente i subscriber che sono interessati ai suoi eventi, e li avvisa quando si verifica un evento

11.9.2 Diagramma delle Classi



11.9.3 Diagramma di Sequenza



12. TDD - Test Driven Development

12.1 Definizione

È una pratica che prevede di scrivere il test unitario prima del codice di produzione.

Il programmatore scrive il test immaginando che **l'interfaccia esista già**, e solo successivamente scrive il codice per farlo passare.

12.2 Cosa è un test unitario?

Un test unitario è un test che si concentra sulla verifica di singole classi e metodi all'interno del codice.

12.2.1 Schema di test

1. **Preparazione:**

Si crea l'oggetto da verificare, chiamato anche fixture

2. **Esecuzione:**

Si eseguono specifiche azioni sulla fixture per attivare le operazioni che si desidera testare

3. **Verifica:**

Si controlla che i risultati ottenuti dal codice corrispondano ai risultati previsti

4. **Rilascio:**

Una fase opzionale in cui si ripuliscono o si rilasciano gli oggetti e le risorse che sono state utilizzate durante il test

12.2.2 Tipi di test

- Test di unità
- Test di integrazione
- Test di sistema
- Test di accettazione

12.3 Vantaggi

- **I test vengono effettivamente scritti:**

Obbligando a scrivere il test prima del codice, si ha la garanzia che i test non vengano tralasciati

- **Maggiore coerenza:**

La soddisfazione del programmatore nel far passare i test porta a una scrittura dei test più coerente e disciplinata

- **Chiarezza progettuale:**

Scrivere il test immaginando l'uso dell'oggetto aiuta a chiarire i dettagli dell'interfaccia e del comportamento atteso prima ancora di implementarli

- **Verifica continua:**

Si ottiene un sistema di verifica automatica, che è costantemente ripetibile e dimostrabile in qualsiasi momento

- **Fiducia nel cambiamento:**

La presenza di test automatici infonde fiducia nel programmatore quando è necessario modificare o fare refactoring del codice sapendo che eventuali errori verranno subito rilevati

12.4 Svantaggi

- Complessità di applicazione
- Curva di apprendimento ripida
- Maggiore tempo di sviluppo iniziale
- Aumento del codice da mantenere

12.5 Progettazione degli input

Per testare in modo efficiente, gli input vanno raggruppati in "partizioni di equivalenza" (insiemi di dati trattati allo stesso modo dal codice).

È fondamentale testare sia input validi che input deliberatamente sbagliati, prestando particolare attenzione ai valori ai limiti (o di frontiera), forzando errori o testando valori critici come puntatori nulli e zeri.

13. Refactoring

13.1 Definizione

È un **metodo strutturato** e disciplinato per riscrivere o ristrutturare del codice esistente senza modificarne il comportamento esterno.

Si tratta di un **processo iterativo** che procede applicando piccoli passi di trasformazione, costantemente supportati dalla ripetizione dei test dopo ciascuna modifica per assicurarsi di non aver "rotto" nulla.

13.2 Obiettivi e Vantaggi

L'utilità principale del refactoring è il miglioramento continuo del codice e la preparazione del sistema a futuri cambiamenti.

Tra le attività e gli scopi principali ci sono:

- Rimuovere il codice duplicato
- Migliorare la chiarezza e la leggibilità generale
- Abbreviare metodi troppo lunghi

13.3 Quando usarlo?

- Seguendo la "Regola del 3"
 1. **La prima volta** che scrivi una porzione di codice, limitati a scriverla per far funzionare la logica
 2. **La seconda volta** che hai bisogno di fare la stessa cosa, potresti semplicemente copiare e incollare il codice
 3. **La terza volta** che ti trovi a duplicare o implementare di nuovo quello stesso pezzo di codice, scatta la regola: devi fermarti e applicare il refactoring
- Si aggiunge una nuova funzionalità
- Si corregge un bug
- Durante la revisione del codice

13.4 Procedura

Il refactoring si esegue seguendo un ciclo preciso e ripetitivo:

1. **Assicurarsi che i test passino**, prima di toccare il codice, tutto deve funzionare
2. **Trovare un code smell**
3. **Determinare come migliorare** questo codice
4. **Effettuare le migliorie**

5. Eseguire nuovamente i test

6. Ripetere il ciclo

13.5 Cosa è un Code Smell?

Un Code Smell è un sintomo o un indicatore presente nel codice sorgente che suggerisce la necessità di applicare una tecnica di refactoring per migliorarne la progettazione.

Non si tratta di un bug o di un errore che impedisce l'esecuzione del programma, ma piuttosto di un difetto strutturale o di design che rende il software difficile da capire, da modificare e da riutilizzare nel tempo

13.6 Tabella Code Smell

Nome	Sintomi	Problema	Soluzione
Long Method	Un metodo contiene troppe righe di codice (spesso più di dieci).	I metodi lunghi sono molto difficili da capire, modificare, testare e riutilizzare.	Applicare <u>Extract Method</u> . Se ci sono troppe variabili locali, usare <u>Replace Temp with Query</u> o <u>Introduce Parameter Object</u> . Per i condizionali usare <u>Decompose Conditional</u> .
Duplicated Code	Lo stesso frammento di codice (o molto simile) appare in più punti del programma.	Se c'è un bug, va corretto in tutte le copie. Aumenta inutilmente la dimensione e la complessità del codice.	Applicare <u>Extract Method</u> per isolare il codice duplicato in un unico punto e sostituire i cloni con la chiamata a tale metodo.
Feature Envy	Un metodo accede continuamente ai dati di un'altra classe piuttosto che a quelli della propria.	Errore di progettazione: il metodo si trova nella classe sbagliata e viola il principio di incapsulamento.	Usare <u>Move Method</u> per spostare l'intero metodo, oppure <u>Extract Method</u> per isolare la parte "invidiosa" e spostarla nella classe che possiede i dati.
Large Class	Una classe contiene troppi campi, metodi o un numero eccessivo di linee di codice.	Violazione della coesione: la classe sta chiaramente gestendo più di una singola responsabilità.	Separare le responsabilità creando nuove classi mirate tramite <u>Extract Class</u> o, se ha senso gerarchicamente, <u>Extract Subclass</u> .
Switch Statement	Presenza di istruzioni <code>switch</code> o lunghe catene di <code>if-else</code> basate sul tipo di un oggetto.	La stessa struttura condizionale tende a ripetersi nel codice. Se si aggiunge un nuovo tipo, bisogna modificare tutti gli <code>switch</code> .	Sostituire la logica condizionale sfruttando il polimorfismo con <u>Replace Conditional with Polymorphism</u> .

Tabella 13.1

Nome	Sintomi	Problema	Soluzione
Data Class	Una classe contiene solo attributi e i rispettivi metodi getter/setter, senza logica.	È un contenitore passivo che spinge le altre classi a soffrire di <u>Feature Envy</u> , tipico di una programmazione procedurale e non a oggetti.	Cercare chi utilizza questi dati e migrare quelle funzionalità dentro la Data Class usando <u>Move Method</u> o <u>Extract Method</u> .
Long Parameter List	Un metodo richiede il passaggio di un numero eccessivo di parametri.	Firme dei metodi difficili da leggere e utilizzare, che tendono a cambiare spesso nel tempo.	Raggruppare i parametri correlati in un oggetto tramite <u>Introduce Parameter Object</u> , oppure passare l'intero oggetto con <u>Preserve Whole Object</u> .
Shotgun Surgery	Per completare una singola modifica funzionale, si devono fare tanti piccoli interventi sparsi in molte classi diverse.	Una singola responsabilità logica è stata frammentata e distribuita in modo errato su troppe classi.	Riunire i comportamenti correlati in un'unica classe usando <u>Move Method</u> e <u>Move Field</u> . Se alcune classi si svuotano, usare <u>Inline Class</u> .
Comment	Un blocco di codice o un metodo richiede un'enorme quantità di commenti per essere compreso.	Il codice non è auto-esplicativo e risulta scritto male. I commenti servono a compensare la scarsa chiarezza.	Estrarre le parti complesse con <u>Extract Variable</u> o <u>Extract Method</u> , assegnando nomi chiari (<u>Rename Method</u>) che rendano inutile il commento.

14. Visibilità

14.1 Definizione

È la capacità di un oggetto di "**vedere**" o di **possedere un riferimento** a un altro oggetto.

Questa condizione è requisito strutturale essenziale: affinché un oggetto mittente possa inviare un messaggio a un oggetto destinatario, il destinatario deve rientrare nella portata del mittente, ovvero essergli visibile.

14.2 Tipi di visibilità

La visibilità da un oggetto A a un oggetto B può essere ottenuta in 4 modi comuni.

14.2.1 Visibilità per attributo

Si verifica quando l'oggetto B è dichiarato come un attributo all'interno della classe dell'oggetto A.

Questa è considerata una visibilità relativamente **permanente**, poichè il riferimento e il collegamento persistono per tutto il tempo in cui gli oggetti esistono.

14.2.2 Visibilità per parametro

Avviene quando l'oggetto B viene passato come parametro all'interno di un metodo dell'oggetto A.

Si tratta di una visibilità relativamente **temporanea**, in quanto è valida ed esiste esclusivamente per la durata dell'ambito del metodo in cui è stata passata

14.2.3 Visibilità locale

Si manifesta quando l'oggetto B è dichiarato come un oggetto locale all'interno di un metodo di A.

Anch'essa è una visibilità relativamente **temporanea** limitata all'ambito del metodo.

Può essere ottenuta in 2 modi:

- Creando una nuova istanza locale e assegnandola a una variabile
- Assegnando a una variabile locale l'oggetto restituito dalla chiamata a un altro metodo

14.2.4 Visibilità globale

Si ottiene quando l'oggetto B è dichiarato in modo da essere visibile globalmente ad A.

È una visibilità relativamente **permanente** e si realizza assegnando un'istanza a una variabile globale

14.3 Trasformazione della visibilità

La visibilità **non è un concetto statico**, ma le diverse forme di visibilità possono mutare ed essere convertite l'una nell'altra durante l'esecuzione nel codice.

14.3.1 Da visibilità locale a visibilità per parametro

Si verifica quando un oggetto istanziato o recuperato localmente all'interno di un metodo (visibilità locale) viene passato come argomento (parametro) nella chiamata a un altro metodo.

14.3.2 Da visibilità per parametro a visibilità per attributo

Da visibilità per parametro a visibilità per attributo: È una pratica comune nei metodi di inizializzazione, come i costruttori.

Un oggetto viene passato in ingresso a un metodo come parametro (visibilità temporanea) e, all'interno di quel metodo, viene assegnato a una variabile di istanza della classe, trasformandosi a tutti gli effetti in un attributo stabile nel tempo.